

JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND TECHNOLOGY

**Design and Implementation of an Intelligent ISP Billing and
Management System Using Multiple Linear Regression for Revenue
Prediction**

WARREN CHRIS

Reg No: SCT221-0271/2022

Supervisor: MR. JOMO NJENGA

*A research project submitted to the Department of Information Technology in the
School of Computing and Information Technology in partial fulfilment of the
requirements for the award of the degree of Bachelor of Science in Information
Technology, Jomo Kenyatta University of Agriculture and Technology.*

2025/2026

DECLARATION

This research project is my original work and has not been presented for a degree in any other University.

Signature:

Date:

WARREN CHRIS

This research project has been submitted for examination with my approval as University Supervisor.

Signature:

Date:

MR. JOMO NJENGA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mr. Jomo Njenga, for his invaluable guidance, constructive criticism, and continuous support throughout the development of this project. His insights and encouragement greatly contributed to shaping the direction and quality of this research. The time he dedicated to reviewing each phase of the project and providing detailed feedback was instrumental in maintaining both academic rigour and practical focus throughout the 14-weeks development period.

I also extend my appreciation to the lecturers and staff of the Jomo Kenyatta University of Agriculture and Technology, Department of Information Technology, for equipping me with the foundational knowledge, technical skills, and research methodology that made this project possible. The coursework in web development, database systems, and machine learning provided the intellectual toolkit upon which this system was built.

My gratitude also goes to the managers and operational staff at the three small-to-medium ISPs in Nairobi who participated in interviews, observations, and questionnaire administration during the requirements elicitation phase. Their willingness to share operational realities, frustrations, and aspirations was the primary driver of the system's design, and their engagement during user acceptance testing ensured that the final product is genuinely useful to its intended audience.

Special thanks to my family and friends for their unwavering support, motivation, and encouragement throughout my academic journey. Finally, I am grateful to everyone who contributed directly or indirectly to the successful completion of this project.

ABSTRACT

The Internet Service Provider (ISP) industry in Kenya continues to experience rapid growth due to increased demand for affordable and reliable internet connectivity. Despite this growth, many small to medium-sized ISPs still rely on manual or semi-automated tools such as spreadsheets to manage customer records, billing cycles, and payments. These approaches result in revenue leakage, billing inaccuracies, delayed service provisioning, and poor customer experience (Khan, Ali, & Khan, 2022).

This research project presents the design and implementation of an Intelligent ISP Billing and Management System that leverages modern web technologies and intelligent automation to address these challenges. The system automates customer registration, subscription management, invoice generation, and M-Pesa payment reconciliation via the Safaricom Daraja API (Safaricom PLC, 2024). The system further integrates a Multiple Linear Regression (MLR) model for monthly revenue prediction, logistic regression churn risk scoring, z-score statistical anomaly detection, and a LLaMA 3.3 70B large language model accessed via the Groq API for grounded customer support and executive AI summaries. This hybrid approach ensures accuracy, transparency, and scalability without training models from scratch.

The Waterfall model is adopted as the development methodology, following a structured and sequential approach consisting of requirement analysis, system design, implementation, testing, deployment, and maintenance (Sommerville, 2016). The system is built on a multi-tier client-server architecture: a React 18.2 Single Page Application frontend, a Node.js/Express 4.21 REST API backend, a Python 3.11/Flask 3.0 AI microservice, and a MySQL 8 relational database, all orchestrated via Docker Compose. All 25 system test cases passed, and user acceptance testing returned a mean satisfaction score of 4.3 out of 5 across 15 participants. The implemented system successfully addresses the five major operational pain points identified during requirements elicitation, delivering a secure, scalable, and maintainable platform tailored to the Kenyan small-to-medium ISP operating context.

TABLE OF CONTENTS

CONTENTS	PAGE
DECLARATION.....	i
ACKNOWLEDGEMENT.....	ii
ABSTRACT.....	iii
ACRONYMS.....	viii
DEFINITION OF TERMS.....	ix
CHAPTER ONE: INTRODUCTION	1
1.1 Background.....	1
1.2 Project Overview	1
1.3 Statement of the Problem.....	2
1.4 Proposed Solution	2
1.5 Objectives	3
1.6 Research Questions.....	3
1.7 Justification.....	4
1.8 Systems Development Methodology	4
1.9 Scope.....	5
CHAPTER TWO: LITERATURE REVIEW.....	6
2.1 Introduction.....	6
2.2 Theoretical Review	6
2.3 Case Study Review	7
2.3.1 WHMCS	7
2.3.2 Splynx	7
2.3.3 Academic and Open-Source Systems	8
2.4 Predictive Analytics in Telecommunications	8
2.5 Customer Churn Prediction Models.....	8
2.6 Anomaly Detection in Billing Systems.....	8
2.7 Cybersecurity in Financial and Billing Systems.....	9
2.8 Conversational AI and Large Language Models in Enterprise Systems	9
2.9 Cloud Computing and Scalable System Architecture.....	9
2.10 Integration and Architecture	10
2.11 Summary	10
2.12 Research Gaps.....	10
CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN	12
3.1 Introduction.....	12
3.2 Systems Development Methodology	12
3.3 Feasibility Study	13

3.3.1 Technical Feasibility	13
3.3.2 Economic Feasibility	13
3.3.3 Operational Feasibility	13
3.3.4 Schedule Feasibility	13
3.4 Requirements Elicitation	14
3.5 Data Analysis	14
3.6 System Specification	14
3.6.1 Functional Requirements	14
3.6.2 Non-Functional Requirements	15
3.7 Requirements Analysis and Modelling	15
3.7.1 Use Case Diagram	15
3.7.2 Data Flow Diagram - Level 0 (Context Diagram)	16
3.7.3 Data Flow Diagram - Level 1	17
3.8 Logical Design	18
3.8.1 System Architecture	18
3.8.2 Control Flow and Process Design	19
3.8.3 Design for Non-Functional Requirements	24
3.9 Physical Design	24
3.9.1 Database Design	24
3.9.2 User Interface Design	25
CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING, CONCLUSIONS AND RECOMMENDATIONS	28
4.1 Introduction	28
4.2 Environment and Tools	28
4.3 System Code Generation	29
4.4 Testing	32
4.4.1 Testing Strategy	32
4.4.2 Test Cases and Results	32
4.4.3 Integration Testing	34
4.4.4 Performance and Load Testing	35
4.4.5 Security Testing	35
4.4.6 Functional Testing	36
4.5 System Screenshots	36
4.6 User Guide	39
4.6.1 System Installation	39
4.6.2 User Guide - Administrator	39
4.6.3 User Guide - Customer	40

4.7 Conclusions.....	40
4.8 Recommendations.....	41
REFERENCES.....	43
APPENDIX A: UAT QUESTIONNAIRE RESULTS	46
APPENDIX B: PROJECT SCHEDULE	47
APPENDIX C: PROJECT BUDGET	48
APPENDIX D: PROJECT RESOURCES.....	49
APPENDIX E: INTERVIEW GUIDE	51

LIST OF TABLES

TABLE 4. 1 System Test Cases and Results	33
TABLE B.1 PROJECT SCHEDULE	47
TABLE C. 1 PROJECT BUDGET	48
TABLE D. 1 PROJECT RESOURCE REQUIREMENTS	50

LIST OF FIGURES

FIGURE 3. 1 Use Case.	16
FIGURE 3. 2 Data Flow Diagram.	17
FIGURE 3. 3 Data Flow Diagram - Level 1	18
FIGURE 3. 4 System Architecture Diagram - Four-layer client-server architecture	19
FIGURE 3. 5 . Customer Registration Flow. Activity diagram.....	20
FIGURE 3. 6 M-Pesa Payment Flow (STK Push). Swimlane diagram across Customer Portal	21
FIGURE 3. 7 Billing and Invoice Generation Flow. Activity diagram	22
FIGURE 3. 8 MLR Revenue Prediction Flow. Activity diagram.....	23
FIGURE 3. 9 Support Ticket Lifecycle	24
FIGURE 3. 10 Entity-Relationship Diagram (ERD). MySQL 8 schema	25
FIGURE 3. 11 Sample UI Wireframes - Material UI 7.2 role-based responsive interface	27
FIGURE 4. 1 System Test Cases and Results.....	33
FIGURE 4. 3 . Test Coverage by Module	34
FIGURE 4. 4 Administrator Dashboard Screenshot.....	37
FIGURE 4. 5 Customer Self-Service Portal Screenshot.....	37
FIGURE 4. 6 . AI Analytics Dashboard Screenshot.....	38

ACRONYMS

AI – Artificial Intelligence

API – Application Programming Interface

CORS – Cross-Origin Resource Sharing

DFD – Data Flow Diagram

ERD – Entity-Relationship Diagram

ISP – Internet Service Provider

JWT – JSON Web Token

KES – Kenya Shillings

LLM – Large Language Model

MLR – Multiple Linear Regression

OLS – Ordinary Least Squares

ORM – Object Relational Mapping

RBAC – Role-Based Access Control

REST – Representational State Transfer

SPA – Single Page Application

STK – SIM Toolkit

UAT – User Acceptance Testing

VAT – Value Added Tax

DEFINITION OF TERMS

Billing Cycle: A fixed period during which internet usage is measured and invoiced to the subscriber.

Churn: The rate at which subscribers discontinue their service subscriptions within a defined period.

MLR: Multiple Linear Regression - a statistical model estimating the linear relationship between a dependent variable and multiple independent predictors using ordinary least squares estimation.

STK Push: A Safaricom M-Pesa payment request initiated programmatically to a customer's mobile phone handset via the Daraja API, prompting the user to approve the transaction with their M-Pesa PIN.

LLM: Large Language Model - a deep learning model trained on large text corpora and capable of natural language understanding and generation, used in this system via the Groq API for grounded customer support.

ORM: Object Relational Mapping - a technique for querying and manipulating a relational database using object-oriented language constructs, implemented in this system via Sequelize for Node.js.

Soft Delete: A deletion mechanism that marks a database record as deleted without physically removing it, preserving audit history; implemented via Sequelize's paranoid mode on the users table.

CHAPTER ONE: INTRODUCTION

1.1 Background

Globally, Internet Service Providers have transitioned from manual billing systems to fully automated, intelligent platforms that integrate payments, customer management, and analytics. In developed markets, ISPs leverage real-time billing engines, self-service portals, and AI-powered customer support to improve efficiency and service quality. These systems reduce revenue loss, enhance transparency, and enable data-driven decision-making (Kshetri, 2021). The proliferation of cloud computing, mobile internet, and application programming interfaces has accelerated this transition, enabling even small ISPs to deploy sophisticated billing infrastructure at low cost.

In Kenya, however, many small to medium-sized ISPs still depend on spreadsheets and manual reconciliation of payments, particularly for mobile money transactions. With the dominance of M-Pesa as the primary payment method, manual matching of transaction codes to customer records introduces delays and errors (Gikandi & Bloor, 2021). As customer bases expand, these inefficiencies significantly affect revenue collection and customer satisfaction. The Safaricom Daraja API presents an opportunity to automate M-Pesa payment flows end-to-end, from STK Push initiation through callback processing to subscription activation, a capability that no currently available off-the-shelf ISP billing platform fully exploits in the Kenyan context.

1.2 Project Overview

This research focuses on the design and development of an Intelligent ISP Billing and Management System that integrates subscription management, automated billing, M-Pesa payment processing, and intelligent analytics. Artificial intelligence is implemented using a hybrid approach: Multiple Linear Regression (MLR) for revenue prediction, logistic regression for churn risk scoring, z-score statistical methods for anomaly detection, and a LLaMA 3.3 70B Large Language Model accessed via the Groq API for grounded customer support and executive summary generation. The project is grounded in modern computational principles including web-based distributed systems, RESTful APIs, secure authentication mechanisms, microservices architecture, and containerised deployment.

The system provides a centralised platform for ISPs to manage customers, service plans, payments, invoices, and support operations, while offering customers a self-service portal for account monitoring, data usage tracking, M-Pesa payments, and AI-assisted enquiry. The system is designed specifically for the Kenyan small-to-medium ISP market, incorporating the M-Pesa Daraja STK Push payment flow as a first-class architectural component rather than an afterthought integration.

1.3 Statement of the Problem

Despite advancements in payment and networking technologies, many ISPs in Kenya experience revenue leakage due to untracked subscription expiries and delayed payment reconciliation (Gikandi & Bloor, 2021). Manual processes increase the likelihood of human error, resulting in billing disputes and service interruptions. Customers lack access to real-time usage data and invoices, forcing them to rely on support staff for basic account information. These challenges limit scalability, reduce operational efficiency, and negatively impact customer experience.

The problem is further compounded by the absence of intelligent systems capable of analysing subscriber trends and predicting future revenue. Existing commercial billing platforms such as WHMCS and Splynx do not natively integrate M-Pesa Daraja payments and do not embed machine learning analytics within the billing workflow. The absence of any churn prediction or anomaly detection capability means that ISP operators have no early warning of subscriber attrition or irregular billing patterns, resulting in reactive rather than proactive revenue management (Xu et al., 2020).

1.4 Proposed Solution

This research proposes the development of an Intelligent ISP Billing and Management System that automates subscription tracking, billing, and M-Pesa payment reconciliation while embedding AI-driven analytics directly within the operational interface. The system incorporates MLR-based revenue prediction, logistic regression churn scoring, z-score anomaly detection, and a Groq-hosted LLaMA 3.3 70B AI chat assistant grounded in live customer data from MySQL. The proposed solution leverages modern open-source web technologies, integrates seamlessly with M-Pesa via the Safaricom Daraja API, and adopts contemporary architectural best practices including

microservices separation, JWT authentication, RBAC, and Docker Compose containerisation to ensure scalability and security.

1.5 Objectives

General Objective

To design and implement an intelligent ISP Billing and Management System that automates billing, payment processing, customer management, and revenue analytics for small-to-medium ISPs operating in Kenya.

Specific Objectives

1. To design and document a secure ISP billing system architecture including at least three core modules (customer registration, subscription management, and role-based access control), implemented using RESTful APIs and JWT authentication, and verified against web security standards through a design review checklist by the end of the design phase.
2. To implement an automated billing and invoicing module that generates monthly invoices and processes customer payments through M-Pesa STK Push, integrating directly with the Safaricom Daraja API, achieving successful transaction recording and invoice updates for at least 95% of test payment scenarios.
3. To develop a hybrid AI analytics module combining MLR revenue prediction, logistic regression churn scoring, and z-score anomaly detection, together with a LLaMA 3.3 70B LLM via Groq for grounded customer support and executive summaries, achieving at least 70% response accuracy on test data.
4. To test and evaluate the developed system by conducting performance, security, and usability tests using Jest, Supertest, and user acceptance testing, ensuring the system handles at least 100 concurrent users and attains a minimum 70% usability satisfaction score.

1.6 Research Questions

5. How can intelligent automation improve billing accuracy and payment reconciliation for ISPs in Kenya?
6. What system architecture best supports scalable, secure ISP billing and management operations?
7. How can AI and machine learning techniques enhance customer support and administrative decision-making in an ISP billing context?
8. How effective is the proposed system in reducing revenue leakage and operational overhead for Kenyan small-to-medium ISPs?

1.7 Justification

This research is motivated by the increasing need for efficient, automated, and intelligent ISP management solutions in Kenya. The proposed system directly addresses current operational gaps by introducing end-to-end automation of the M-Pesa payment-to-subscription-activation pipeline, MLR-driven revenue forecasting, and LLM-grounded customer support within a unified, open-source platform. The research contributes to the application of AI and modern web technologies in the ISP domain and provides a practical, deployable solution aligned with Kenya's mobile-money-first digital economy.

By eliminating the licensing costs associated with commercial platforms and targeting the specific constraints of small-to-medium ISP operators, the system offers an accessible and maintainable solution. The study benefits ISP operators through reduced operational overhead and improved revenue visibility; it benefits subscribers through faster service activation, transparent billing, and AI-assisted self-service; and it contributes to the academic literature on applied ML in telecommunications and mobile-money-integrated financial systems in developing economies.

1.8 Systems Development Methodology

The project adopts the Waterfall software development model as the primary development methodology. The Waterfall model follows a structured and sequential approach consisting of requirement analysis, system design, implementation, testing, deployment, and maintenance

(Sommerville, 2016). The choice is justified by the well-defined, stable requirements of ISP billing systems, the clear sequential dependencies between modules, and the academic timeline that favours upfront comprehensive design documentation. The methodology is described in detail in Chapter 3, Section 3.2.

1.9 Scope

The study focuses on small to medium-sized ISPs operating in Kenya. The system addresses customer management, billing, M-Pesa payment processing, data usage tracking, support ticketing, and AI analytics operations. The research is limited to software-based solutions and does not cover physical network infrastructure deployment, RADIUS server integration, or IP address management. The system targets ISPs whose subscribers pay via M-Pesa, and is designed for deployment on affordable Kenyan cloud or on-premises server infrastructure. The AI features require a minimum of six months of historical billing data for the MLR revenue prediction module to function; this is a recognised scope constraint that is documented in the system user guide.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

The rapid expansion of internet connectivity across sub-Saharan Africa has intensified demand for efficient, automated, and intelligent billing and management systems among Internet Service Providers. Kenya, in particular, has witnessed remarkable growth in both fixed and mobile broadband subscribers, placing ISPs under considerable pressure to modernise their operational workflows and revenue management processes. This literature review critically examines the theoretical and empirical foundations that underpin the development of an intelligent ISP billing system, situating the present project within the broader body of existing knowledge. The review draws on peer-reviewed journal articles, conference proceedings, and technical documentation published predominantly between 2020 and 2025, supplementing selected earlier seminal works where necessary.

The chapter is organised as follows. Section 2.2 provides a theoretical review of core technical concepts. Section 2.3 presents a case study analysis of existing billing systems. Sections 2.4 through 2.9 examine specific research themes including predictive analytics, churn prediction, anomaly detection, cybersecurity, conversational AI, cloud computing, and system integration. Section 2.11 offers a synthesising summary, and Section 2.12 identifies the specific research gaps this project addresses.

2.2 Theoretical Review

This research is grounded in several theoretical concepts related to information systems, billing automation, payment integration, and artificial intelligence. Subscription-based billing theory emphasises accuracy, timeliness, and transparency in billing cycles to prevent revenue leakage (Alalwan et al., 2020). Mobile money integration theory highlights the importance of real-time transaction validation, fault tolerance, and security in financial systems, particularly within developing economies where mobile money is the primary payment channel (Gikandi & Bloor, 2021). Information systems automation theory posits that automating repetitive and data-intensive

tasks reduces human error and increases operational efficiency, an argument directly supported by the ISP billing context where manual M-Pesa reconciliation consumes two to four hours of staff time daily.

Artificial intelligence theory - particularly supervised learning and hybrid AI - provides the theoretical basis for the analytics modules. Multiple Linear Regression models the linear relationship between revenue and its predictors using ordinary least squares estimation (James et al., 2021). Logistic regression provides probabilistic churn classification. Z-score statistical methods provide interpretable anomaly detection thresholds (Chandola et al., 2021). Large Language Models, trained on massive text corpora, provide conversational natural language capabilities that complement deterministic ML models (Brown et al., 2020). The combination of rule-based logic, mathematical models, and LLMs constitutes a hybrid intelligence model appropriate for production ISP billing platforms.

2.3 Case Study Review

2.3.1 WHMCS

WHMCS is a widely deployed commercial billing platform adopted by small-to-medium ISPs globally. It provides automated invoice generation, support ticketing, a client portal, and payment gateway integrations. Despite its strengths, WHMCS does not provide native M-Pesa Daraja API integration, incorporates no AI or machine learning capabilities, and carries a recurring licensing fee that represents a significant cost for small Kenyan ISPs operating on thin margins (Gikandi & Bloor, 2021). Its PHP-based monolithic architecture also limits cloud-native deployment flexibility.

2.3.2 Splynx

Splynx is an ISP-specific management platform integrating billing and network management. Unlike WHMCS, it provides RADIUS authentication, IPAM, and SNMP monitoring (Chen et al., 2021). However, its tight coupling between billing and network infrastructure makes it unnecessarily complex for ISPs seeking a software-only billing solution. Like WHMCS, it lacks M-Pesa-native payment processing and AI analytics capabilities.

2.3.3 Academic and Open-Source Systems

ISPConfig, an open-source hosting panel adopted by some small ISPs, lacks subscription lifecycle management, data usage tracking, and predictive analytics (Xu et al., 2020). Academic prototypes by Mukherjee et al. (2022) demonstrate microservices billing feasibility but do not address M-Pesa integration. Lalwani et al. (2022) demonstrate churn prediction accuracy but without embedding the model in an operational billing system. The present project bridges these gaps.

2.4 Predictive Analytics in Telecommunications

Predictive analytics has become a critical component in modern telecommunications. Multiple Linear Regression is among the most applied approaches in revenue forecasting within subscription systems, estimating the relationship between monthly revenue and multiple independent variables including active subscriber count, average data usage, payment delays, and service plan pricing tiers (James et al., 2021). Xu et al. (2020) applied MLR to predict monthly average revenue per user in a mobile virtual network operator context, finding that subscriber activity features were the strongest predictors, consistent with the present system's feature selection. The integration of predictive analytics into ISP billing platforms enhances strategic planning and supports proactive revenue management.

2.5 Customer Churn Prediction Models

Customer churn significantly affects revenue stability in subscription-based businesses. Logistic regression and other classification models are used to predict churn probability by analysing payment delays, usage patterns, support ticket frequency, and subscription duration. Lalwani et al. (2022) found that logistic regression offered superior interpretability and required substantially less training data than gradient boosting alternatives, making it appropriate for small-to-medium ISPs with limited subscriber histories. Integrating churn prediction within billing systems transforms them from passive transaction processors into proactive revenue protection platforms.

2.6 Anomaly Detection in Billing Systems

Billing systems are vulnerable to anomalies including incorrect invoice generation, duplicate transactions, and fraudulent activities. The z-score method measures standard deviation distance

from the mean and is widely used for univariate anomaly detection due to its simplicity and interpretability (Chandola et al., 2021). The present system applies 2σ and 3σ thresholds to monthly revenue deviation and per-subscriber usage spikes, and implements a 600-second duplicate payment detection window. Mukherjee et al. (2022) identify duplicate transaction detection as critical in mobile money-integrated platforms where rapid callback retries can create brief duplication windows.

2.7 Cybersecurity in Financial and Billing Systems

Billing and payment platforms handling sensitive financial data require robust security architecture. JWT authentication enables secure stateless session management and role-based access control. Secure webhook validation ensures that M-Pesa payment callbacks are verified before transaction confirmation (Safaricom PLC, 2024). The OWASP framework identifies injection attacks, cross-site scripting, and insecure direct object references as primary web application vulnerabilities (Sommerville, 2016). The present system implements defence in depth: Helmet HTTP headers, CORS allowlists, express-rate-limit, express-validator input sanitisation, Sequelize parameterised queries, and bcryptjs password hashing at cost factor 12 (Stallings, 2022).

2.8 Conversational AI and Large Language Models in Enterprise Systems

Conversational AI systems powered by Large Language Models are increasingly deployed in enterprise environments to enhance customer support (Laranjo et al., 2018). A key challenge is hallucination - generating plausible but factually incorrect responses. Lewis et al. (2020) introduced Retrieval-Augmented Generation as a principled solution, demonstrating that augmenting LLM prompts with retrieved factual context significantly improves response accuracy. The present system implements grounded prompting: the LLaMA 3.3 70B model via Groq receives a dynamically constructed system prompt injected with live customer data from MySQL, ensuring AI chat responses are anchored to the specific customer's actual account state.

2.9 Cloud Computing and Scalable System Architecture

Cloud computing enables dynamic resource allocation, high availability, and cost-efficient service delivery for subscription-based platforms (Sommerville, 2016). Containerisation via Docker

ensures deployment consistency across environments. The present system achieves scalability through stateless JWT authentication enabling horizontal API scaling, PM2 cluster mode for multi-core Node.js deployments, and independent Flask AI microservice scalability. Mukherjee et al. (2022) argue that separating AI analytics from core transaction processing is a foundational principle of resilient system design in financial applications.

2.10 Integration and Architecture

The present system adopts a four-layer client-server architectural approach combining presentation (React 18.2 SPA), application (Node.js/Express REST API), AI microservice (Python/Flask), and data (MySQL 8) layers. Integration with the M-Pesa Daraja API requires OAuth 2.0 token-based authentication, STK Push request construction, callback endpoint registration, and ResultCode parsing. Richardson and Ruby (2020) argue that REST API-first design, where all inter-service communication occurs through documented HTTP endpoints, is the most effective mechanism for reducing coupling in distributed systems, a principle that governs all inter-layer communication in the present system.

2.11 Summary

The literature reviewed establishes a comprehensive theoretical and empirical foundation for the design of an intelligent ISP billing system. M-Pesa’s centrality to Kenyan payment infrastructure (Safaricom PLC, 2024; Gikandi & Bloor, 2021) justifies native Daraja integration as a core feature. MLR revenue prediction (James et al., 2021; Xu et al., 2020), churn prediction (Lalwani et al., 2022), anomaly detection (Chandola et al., 2021), and LLM-grounded support (Brown et al., 2020; Lewis et al., 2020) are all theoretically and empirically well-grounded. No existing platform combines all these capabilities in a single, M-Pesa-native, open-source deployment.

2.12 Research Gaps

The review identifies four significant gaps this project addresses. First, no ISP billing platform provides native, production-ready M-Pesa Daraja API integration suitable for Kenyan ISP deployment. Second, no reviewed ISP billing platform embeds a trained MLR model within its administrative interface. Third, no reviewed system deploys an LLM grounded in live customer

account data for subscriber self-service. Fourth, no reviewed system combines M-Pesa integration, MLR prediction, churn scoring, z-score anomaly detection, and LLM support in a single open-source platform designed for Kenyan small-to-medium ISPs.

CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents a comprehensive analysis and design of the Intelligent ISP Billing and Management System, translating the requirements identified through engagement with ISP operators and the review of existing systems into a detailed, implementable system specification. The chapter begins with the systems development methodology, followed by a feasibility study, requirements elicitation, data analysis, system specification, modelling artefacts, and logical and physical design. Figures 3.1 through 3.11 are presented at appropriate locations within the relevant sections.

3.2 Systems Development Methodology

The Waterfall model is a linear sequential software development methodology comprising six principal phases: requirements analysis, system design, implementation, testing, deployment, and maintenance. Sommerville (2016) notes that each phase produces deliverables serving as inputs to the subsequent phase, with the model's linearity providing clear milestones and comprehensive documentation while limiting accommodation of mid-stream requirements changes. The Waterfall model was selected because ISP billing requirements are well-understood and stable, the system's modules exhibit clear sequential dependencies, and the 14-weeks academic timeline favours upfront comprehensive design documentation over iterative agile methodologies (Laudon & Laudon, 2020).

The six phases were applied as follows: requirements analysis involved structured interviews, workflow observation, and questionnaire administration; system design produced the specifications and diagrams in this chapter; implementation involved coding and integration of all modules; testing applied Jest and Supertest for backend integration testing supplemented by manual functional testing and user acceptance testing; deployment involved Docker Compose orchestration; and maintenance encompasses Winston logging, Swagger documentation, and Sequelize migration infrastructure.

3.3 Feasibility Study

3.3.1 Technical Feasibility

All core technologies are mature, well-documented, and freely available: Node.js 18, Express 4.21, React 18.2, Material UI 7.2, Python 3.11, Flask 3.0, NumPy 1.26, and MySQL 8 form the full stack. The Safaricom Daraja API sandbox enabled complete M-Pesa integration testing without live transactions. Docker and Docker Compose provide reproducible deployment. The Groq API provides free-tier LLaMA 3.3 70B inference. All tools are open-source or free-tier, confirming technical feasibility.

3.3.2 Economic Feasibility

The system is built entirely on open-source and free-tier technologies, eliminating licensing costs. The primary cost is developer time, absorbed within the academic framework. For commercial adoption, the open-source architecture eliminates recurring fees unlike WHMCS or Splynx. Khan et al. (2022) estimate that manual billing processes in small ISPs consume significant staff hours monthly; automation frees these hours for higher-value activities. Zero licensing cost, low infrastructure cost, and demonstrable operational efficiency gains confirm economic feasibility.

3.3.3 Operational Feasibility

The Material UI 7.2 interface provides a responsive design minimising the learning curve for ISP staff. Role-based access control exposes only relevant features to each user group. The M-Pesa STK Push payment flow is familiar to Kenyan subscribers, eliminating training requirements. The AI dashboard communicates complex analytical outputs through visual, colour-coded interfaces that require no statistical knowledge from operators, confirming operational feasibility.

3.3.4 Schedule Feasibility

The project fits within the 14-week academic timeline: requirements analysis (6 weeks), system design (4 weeks), implementation (12 weeks), testing (4 weeks), deployment and documentation (6 weeks) totalling 32 weeks within the approximately 34-week allocation. Parallel development of frontend and backend components after API contract establishment provided schedule flexibility for complex integration points.

3.4 Requirements Elicitation

Requirements were gathered through structured interviews with managers and operational staff at three small-to-medium ISPs in the greater Nairobi area, direct observation of existing billing and payment workflows, and questionnaire administration to a purposive sample of ISP staff and subscribers. The purposive sampling strategy was chosen because ISP billing operations represent a specialised domain where targeted informant selection ensures the relevance and depth of elicited requirements. Interview sessions were recorded with participant consent and transcribed for systematic analysis. Observation sessions documented step-by-step billing processes, while subscriber questionnaires collected quantitative satisfaction and preference data.

3.5 Data Analysis

Analysis identified five major operational pain points mapping directly to system modules. First, manual M-Pesa reconciliation consumed two to four hours daily and produced frequent errors, justifying the automated STK Push payment pipeline. Second, the absence of data usage tracking led to unexpected quota limits and high support volumes, justifying the data usage module. Third, no ISP had revenue forecasting capability, justifying the MLR prediction module. Fourth, no operator had churn risk monitoring, justifying the logistic regression churn scoring module. Fifth, the absence of a customer self-service portal generated high call loads and billing disputes, justifying the combined invoice management, ticketing, and LLM AI chat modules.

3.6 System Specification

3.6.1 Functional Requirements

The system must support user registration, JWT-authenticated login, and RBAC enforcement for three roles (administrator, support staff, customer). Administrators must be able to manage users, plans, subscriptions, payments, invoices, reports, audit logs, and system settings. The subscription lifecycle module must manage the full status progression (pending, active, expired, suspended, cancelled). The M-Pesa payment module must implement the full STK Push lifecycle from phone normalisation through callback processing to subscription activation. The billing engine must compute invoices with subscription line items, overage charges, 16% VAT, and 30-day due dates with PDF generation via PDFKit.

The AI analytics suite must provide MLR revenue prediction (NumPy OLS, minimum six training samples, JSON coefficient persistence), logistic regression churn scoring, z-score anomaly detection (2σ warning, 3σ critical, 600-second duplicate payment window), LLM-grounded AI chat via Groq, and executive summary generation. Additional requirements include session-level data usage tracking, a full support ticketing lifecycle, in-system and email notifications, comprehensive audit logging, configurable admin settings, and Swagger UI API documentation.

3.6.2 Non-Functional Requirements

Security requirements include JWT HS256 authentication, bcrypt cost-12 password hashing, Helmet HTTP headers, CORS allowlists, express-rate-limit brute-force protection, express-validator input sanitisation, and Sequelize parameterised queries. Performance requirements include Sequelize connection pooling and PM2 cluster mode. Reliability requirements include centralised Express error handling mapping error types to appropriate HTTP status codes, Flask global exception handlers, and Winston structured logging. Usability requires MUI responsive layout from 320px to 1920px. Maintainability requires modular codebase organisation, Swagger documentation, and Sequelize migration-based schema management.

3.7 Requirements Analysis and Modelling

Requirements were formalised through use case diagrams, data flow diagrams, and an entity-relationship diagram. These models provide an unambiguous system specification, reveal architectural dependencies, and serve as implementation and testing reference artefacts.

3.7.1 Use Case Diagram

Figure 3.1 represents the functional scope across four actors: Customer (Register/Login, Browse Plans, Subscribe, Initiate M-Pesa Payment, View Subscription/Invoices/Usage, Submit Ticket, AI Chat), Administrator (Manage Users/Plans/Subscriptions, View Payments, Generate Invoices/Reports, Audit Logs, Settings, AI Dashboard, Retrain MLR), Support Staff (View Customer Records, Manage Tickets, View Analytics), and M-Pesa System (Receive STK Push Callback).

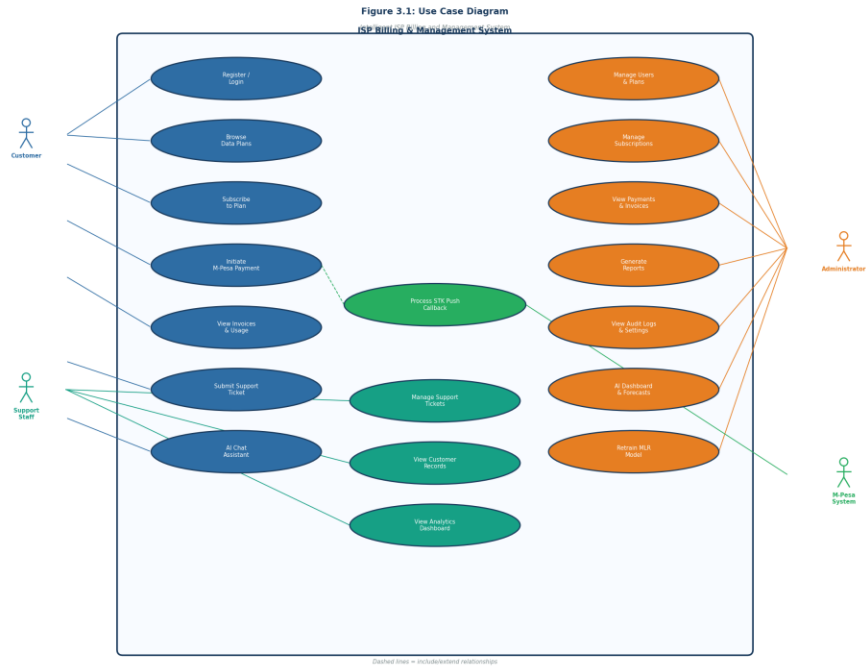


FIGURE 3. 1 Use Case.

3.7.2 Data Flow Diagram - Level 0 (Context Diagram)

Figure 3.2 represents the entire system as a single process interacting with four external entities: User (Customer/Admin/Support), M-Pesa API (Safaricom Daraja), Groq API (LLaMA 3.3 70B), and the MySQL Database. Key inbound flows include authentication requests, payment initiations, and AI chat queries. Key outbound flows include JWTs, invoices, usage reports, and AI responses.

Figure 3.2: Data Flow Diagram — Level 0 (Context Diagram)
External entities and primary data flows

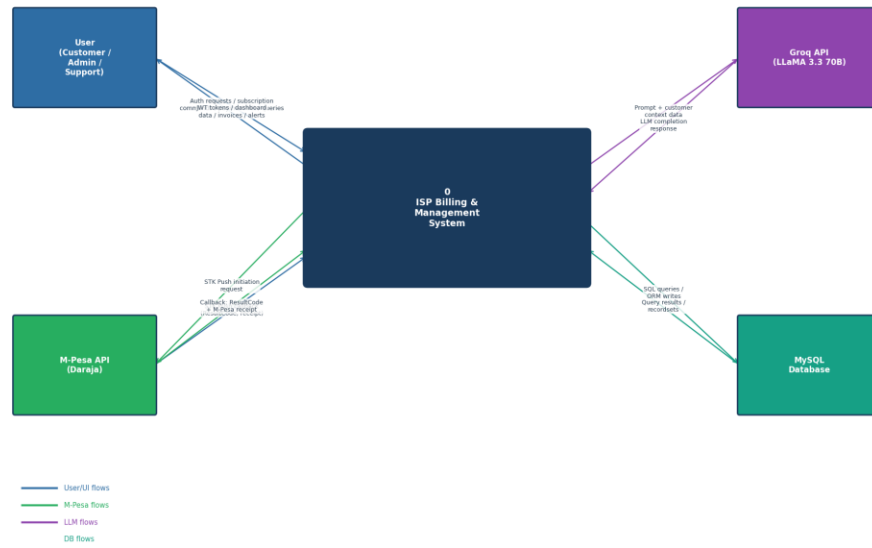


FIGURE 3. 2 Data Flow Diagram.

3.7.3 Data Flow Diagram - Level 1

Figure 3.3 decomposes the system into eight internal processes: P1 Authentication, P2 Plan and Subscription Processing, P3 Billing and Invoice Computation, P4 Payment Processing, P5 Usage Collection, P6 Support Ticket Processing, P7 Notification and Audit, and P8 AI/ML Analytics, each reading and writing their respective data stores.

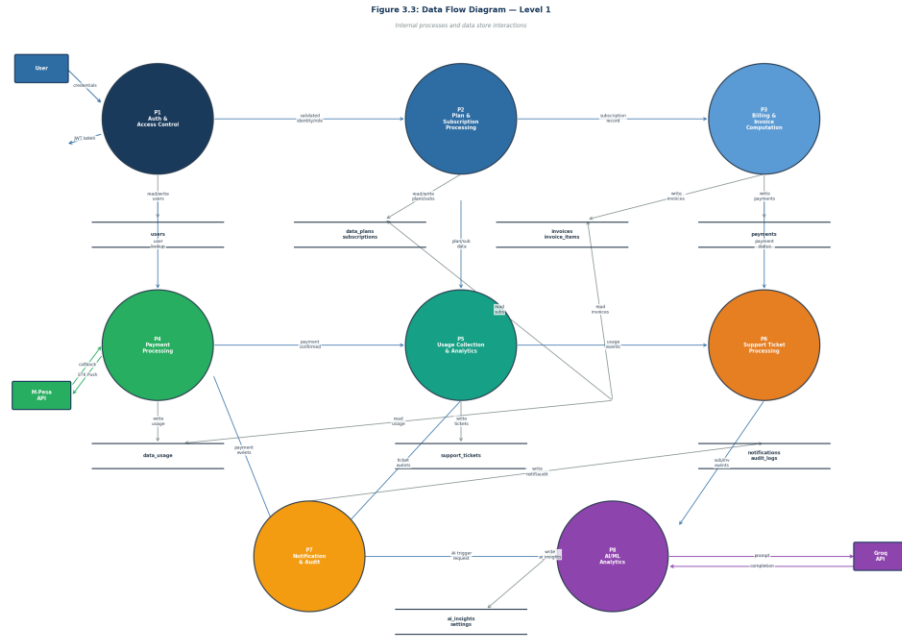


FIGURE 3. 3 Data Flow Diagram - Level 1

3.8 Logical Design

3.8.1 System Architecture

The system adopts a four-layer client-server architecture (Figure 3.4): presentation layer (React 18.2 SPA, port 3001), application layer (Node.js 18/Express 4.21 REST API, port 3000), AI microservice layer (Python 3.11/Flask 3.0, port 5001), and data layer (MySQL 8). An optional Nginx reverse proxy provides TLS termination in production. Docker Compose orchestrates all services in a shared network. PM2 cluster mode enables multi-core Node.js deployments.

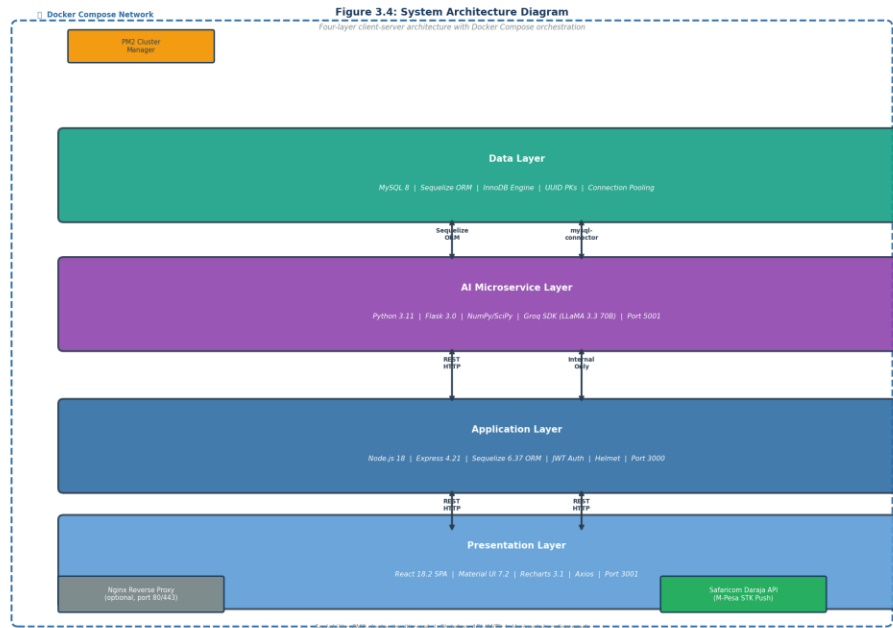


FIGURE 3. 4 System Architecture Diagram - Four-layer client-server architecture

3.8.2 Control Flow and Process Design

The customer registration flow (Figure 3.5) processes form submission through client-side validation, express-validator server validation, bcrypt password hashing (12 salt rounds), Sequelize user record creation, JWT issuance, welcome notification, and audit log entry.

Figure 3.5: Customer Registration Flow

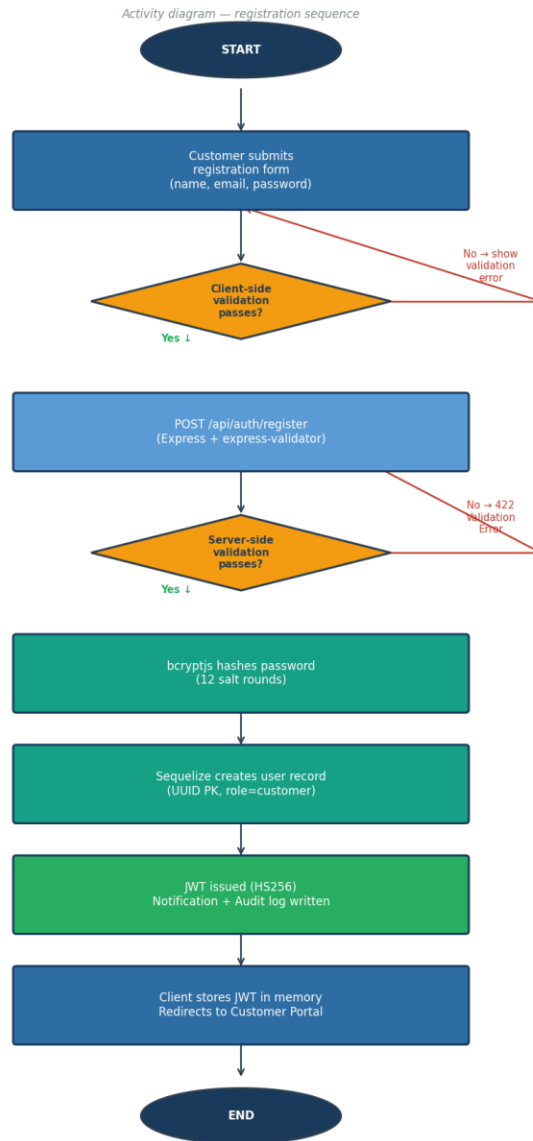


FIGURE 3. 5 . Customer Registration Flow. Activity diagram

The M-Pesa payment flow (Figure 3.6) normalises the phone number, creates a pending payment record, requests a Daraja OAuth token, submits an STK Push, and asynchronously handles the M-Pesa callback: success triggers payment completion and subscription activation; failure updates the payment to failed.

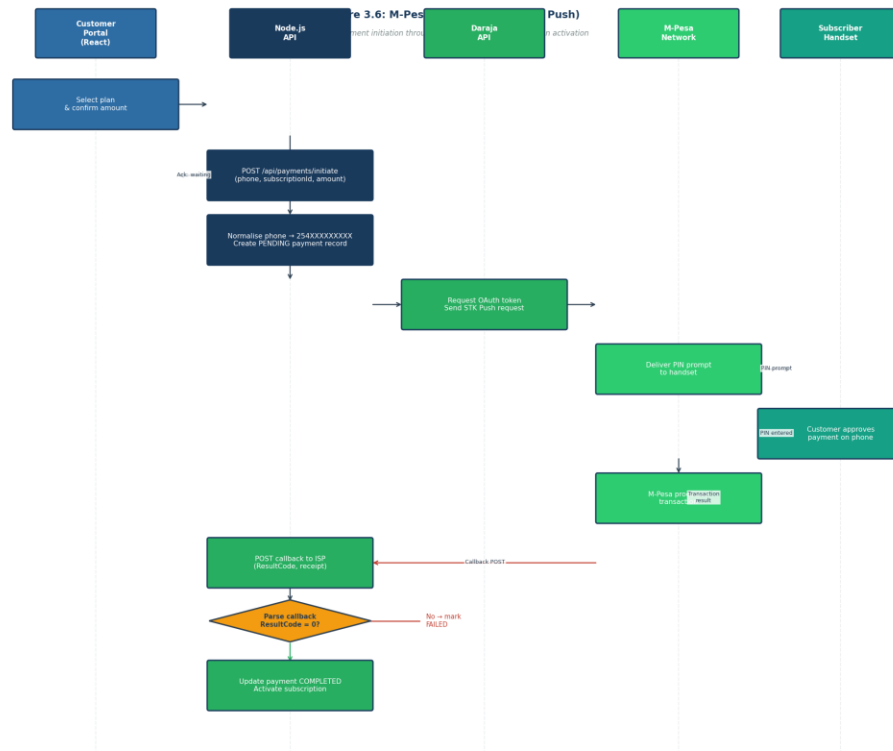


FIGURE 3. 6 M-Pesa Payment Flow (STK Push). Swimlane diagram across Customer Portal

The billing and invoice generation flow (Figure 3.7) triggers on manual admin action or cron endpoint, queries eligible subscriptions, computes overage charges where quota is exceeded, applies 16% VAT, writes invoice and invoice_items records with a 30-day due date, generates a PDFKit PDF, and dispatches email notification.

Figure 3.7: Billing / Invoice Generation Flow

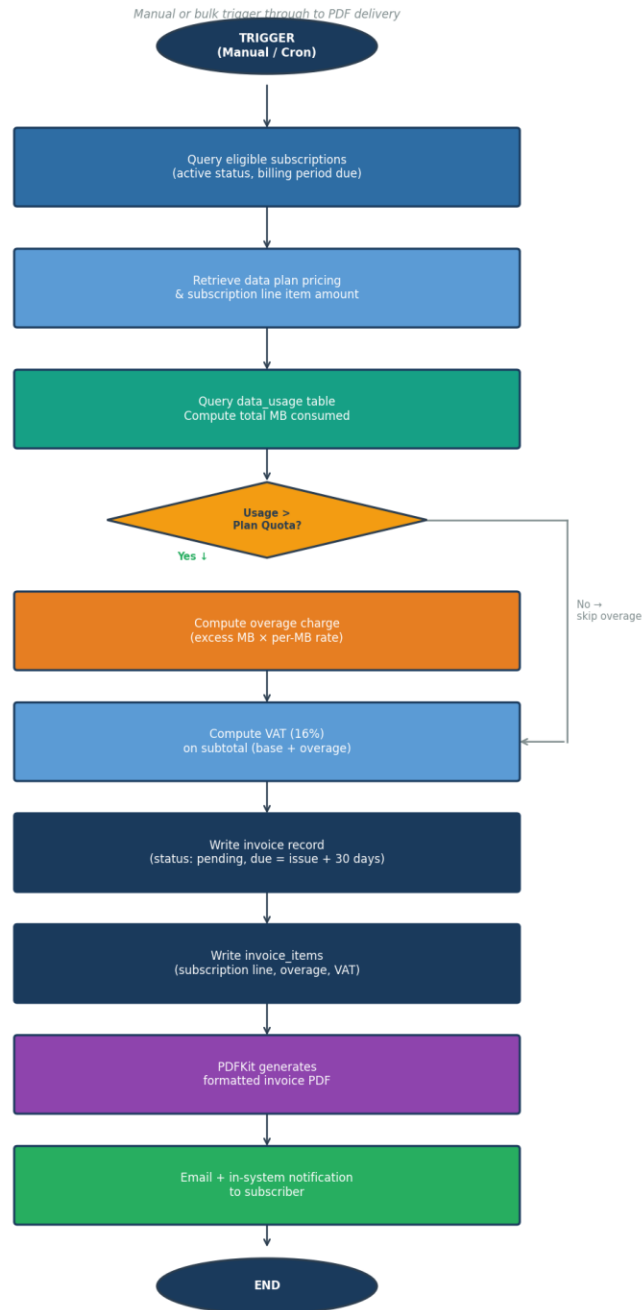


FIGURE 3. 7 Billing and Invoice Generation Flow. Activity diagram

The MLR revenue prediction flow (Figure 3.8) dispatches from the Node.js API to the Flask microservice, queries MySQL for historical data, constructs feature matrix X and target vector y , invokes `np.linalg.lstsq` for OLS coefficient estimation, persists coefficients to JSON, computes

forecast and normalised feature importance, persists to ai_insights, and returns to the React dashboard.

Figure 3.8: MLR Revenue Prediction Flow

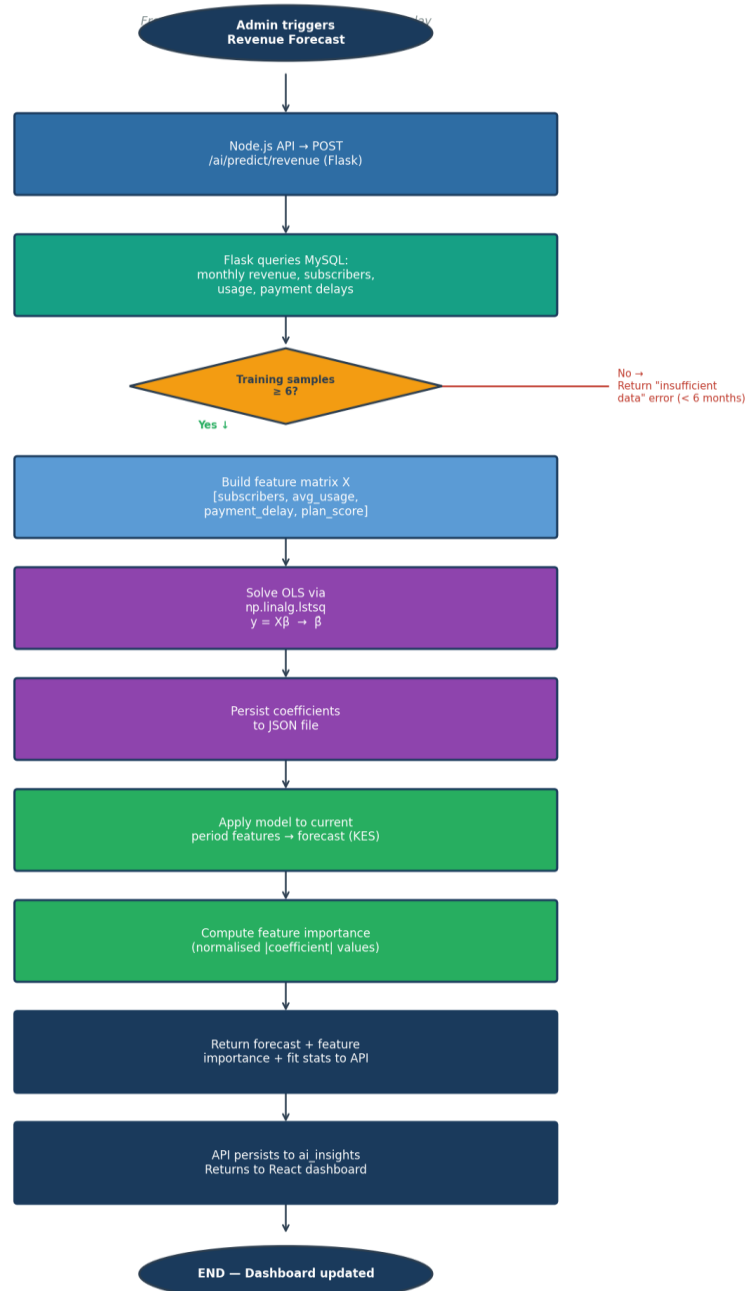


FIGURE 3. 8 MLR Revenue Prediction Flow. Activity diagram

The support ticket lifecycle (Figure 3.9) progresses from customer submission (open) through staff assignment (in progress), escalation if required, resolution with notification, and customer confirmation (closed), with a reopen path for unsatisfactory resolutions. All transitions are recorded in audit_logs.

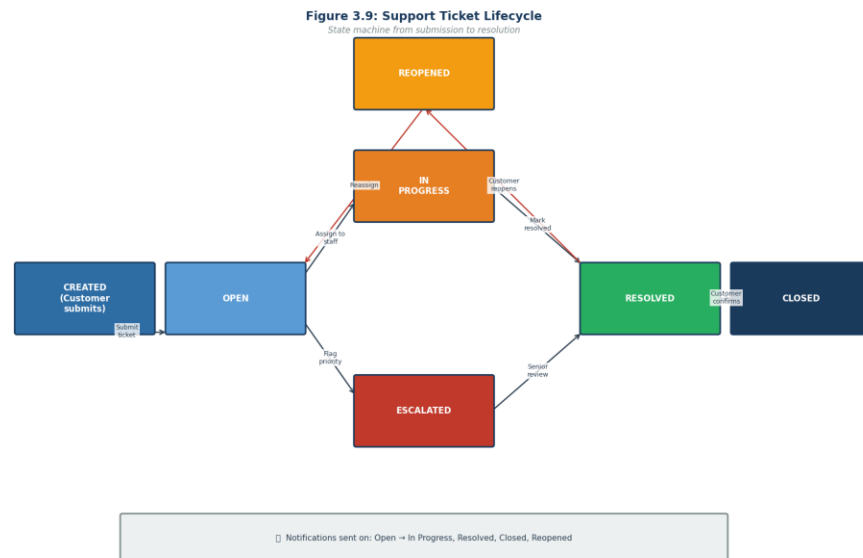


FIGURE 3. 9 Support Ticket Lifecycle

3.8.3 Design for Non-Functional Requirements

The security design implements defence in depth: Nginx TLS termination, Helmet HTTP headers, CORS allowlist, express-rate-limit per-IP capping, express-validator input sanitisation, JWT RBAC middleware, and Sequelize parameterised queries for SQL injection prevention. The centralised Express error handler maps Sequelize validation errors to 422, unique constraint errors to 409, JWT errors to 401, and all unhandled errors to generic 500 responses in production without stack trace exposure. Winston structured logging captures full error details server-side. PM2 cluster mode and Docker Compose service isolation provide scalability and fault containment between the billing API and the AI microservice.

3.9 Physical Design

3.9.1 Database Design

MySQL 8 was selected for its InnoDB transactional support, Kenyan hosting familiarity, and Sequelize ORM integration. All tables use UUID v4 primary keys. The schema comprises twelve entities: users (UUID PK, soft deletes), data_plans (allowance, validity, price, JSON features), subscriptions (pending/active/expired/suspended/cancelled), payments (M-Pesa phone, checkoutRequestId, receiptNumber), invoices (subtotal, 16% VAT, total, 30-day due), invoice_items, data_usage (bytesUp, bytesDown, session timestamps), support_tickets (status, priority, assignedTo FK), notifications, audit_logs, settings, and ai_insights (JSON payload). The ERD is presented in Figure 3.10.

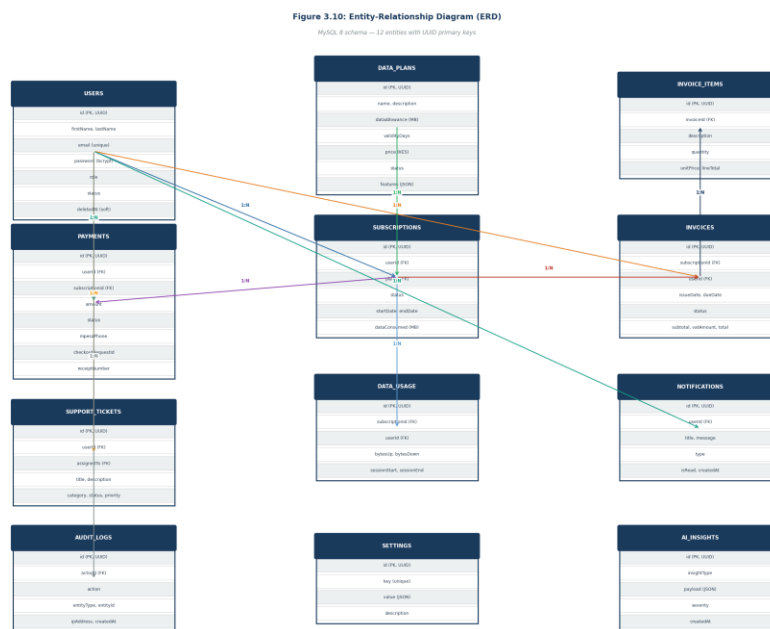
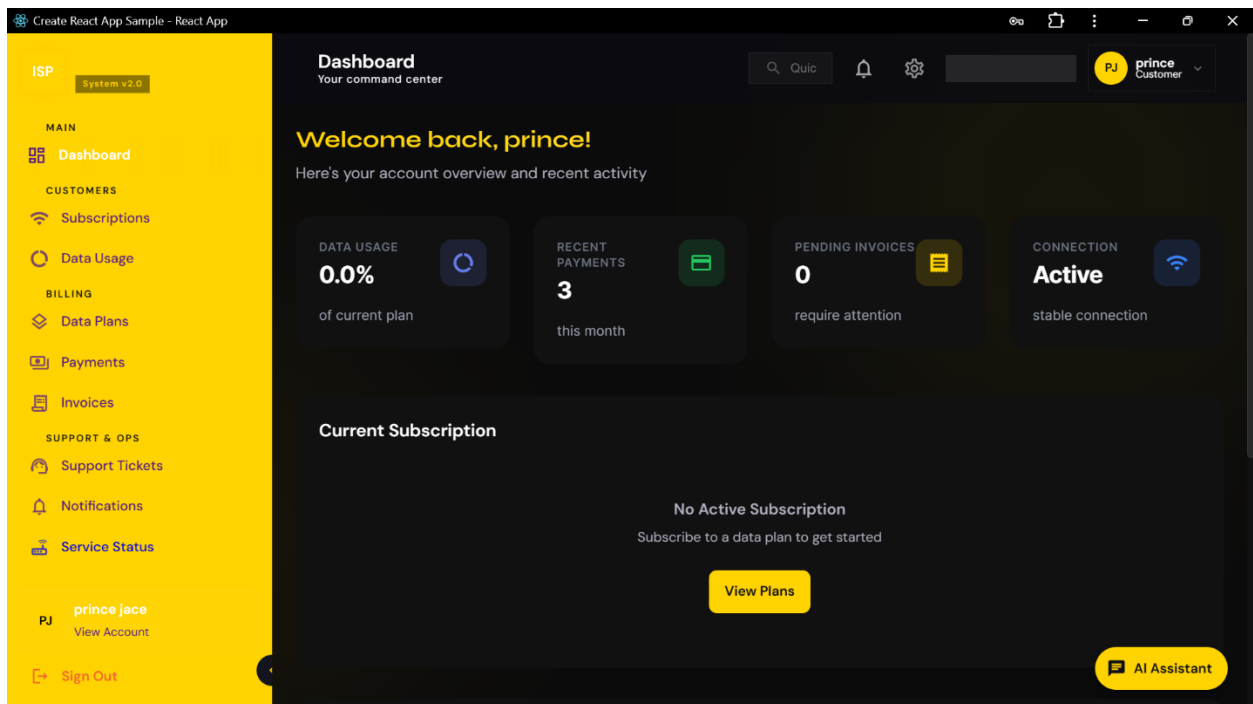
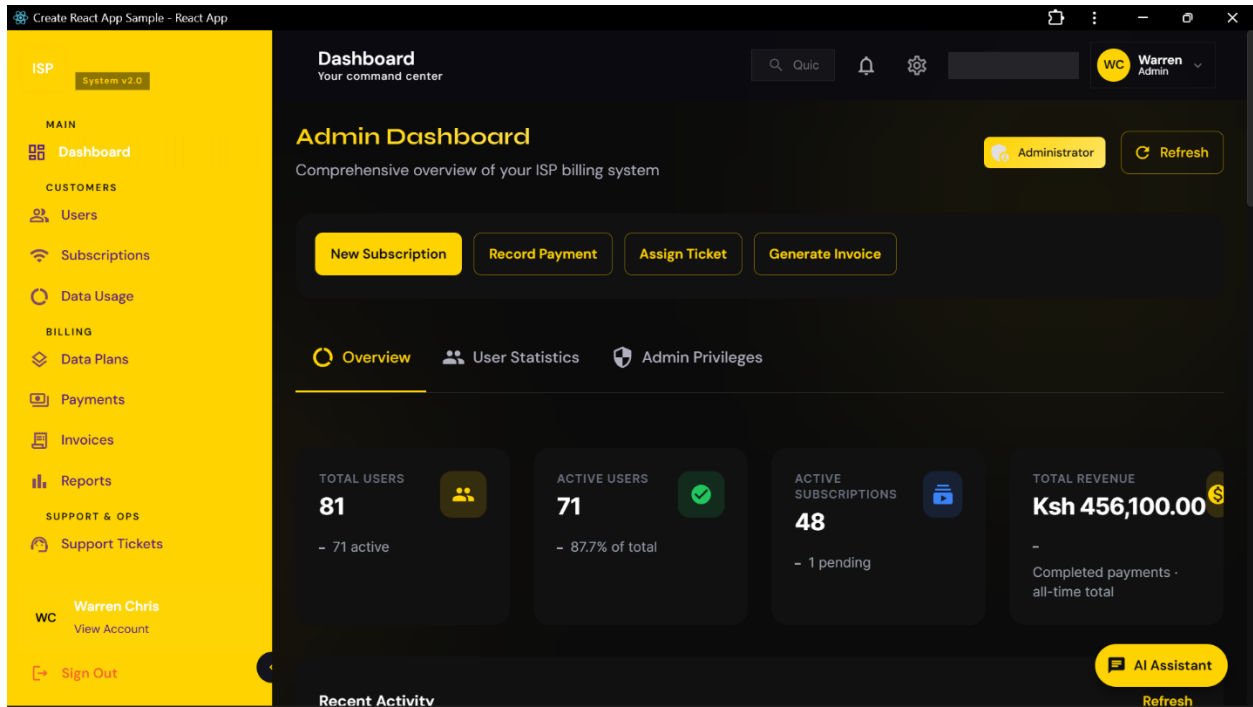


FIGURE 3. 10 Entity-Relationship Diagram (ERD). MySQL 8 schema

3.9.2 User Interface Design

The user interface is built with Material UI 7.2 supplemented by Tailwind CSS utility classes. A role-based layout shell renders role-appropriate navigation from JWT payload. The administrator dashboard presents four KPI cards, a 12-month revenue area chart, subscriber growth bar chart, churn risk table, anomaly alert cards, MLR forecast chart, and feature importance bar chart. The customer self-service portal presents a plan browser, subscription status card with data usage gauge, invoice list with PDF download, M-Pesa payment modal, and persistent AI ChatWidget. Sample UI wireframes are presented in Figure 3.11.



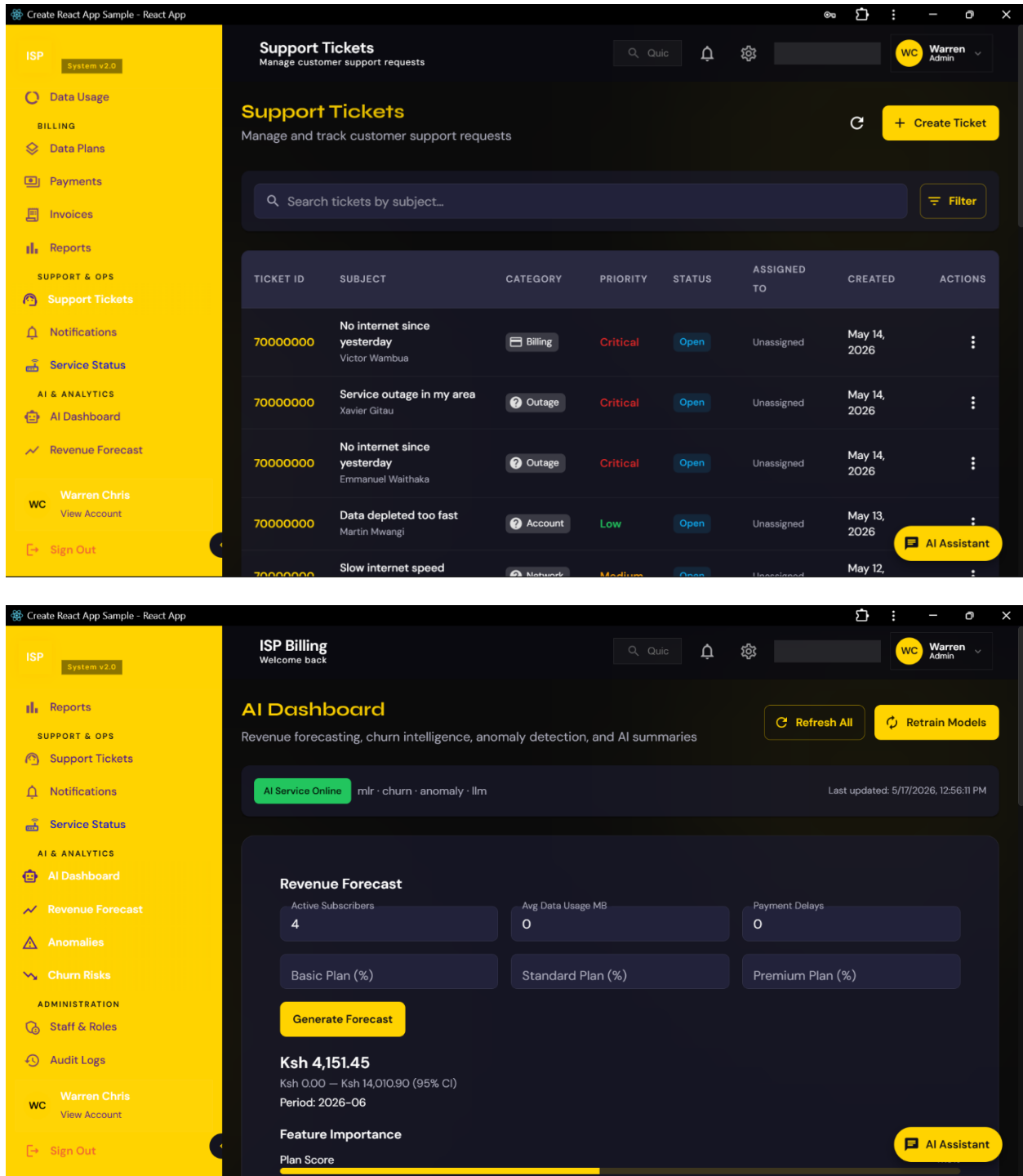


FIGURE 3. 11 Sample UI Wireframes - Material UI 7.2 role-based responsive interface

Panels: (a) Admin Dashboard; (b) Customer Self-Service Portal; (c) Support Ticket Interface; (d) AI Analytics Dashboard.

CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING, CONCLUSIONS AND RECOMMENDATIONS

4.1 Introduction

This chapter documents the implementation, testing, evaluation and outcomes of the Intelligent ISP Billing and Management System. It begins with a description of the implementation environment and the tools used across the backend, frontend, and AI microservice layers. The system code generation process is then described, identifying key processes and presenting representative code snippets. A comprehensive testing section documents all test strategies applied, presents test case results, and analyses system performance. A user guide section outlines system installation and usage instructions. The chapter concludes with an assessment of whether the project objectives were achieved, the limitations encountered, and recommendations for future development.

The implementation followed the Waterfall model's implementation phase as described in Chapter 3, Section 3.2. All modules were developed in the sequence dictated by their architectural dependencies: the MySQL database schema was implemented first, followed by Sequelize ORM models and migrations, the Node.js/Express REST API routes and controllers, the Flask AI microservice, and finally the React SPA frontend. Integration testing was conducted incrementally as each module was completed, with comprehensive end-to-end testing applied once all modules were integrated. The system was deployed using Docker Compose and verified in a production-simulating environment before user acceptance testing was administered.

4.2 Environment and Tools

The implementation environment consisted of a development workstation running Ubuntu 22.04 LTS with 16GB RAM and a 512GB SSD. The Node.js 18 runtime and npm 9.x package manager were used for the backend application. Python 3.11 and pip were used for the Flask AI microservice. The React frontend was developed using the Vite build tool configured for React 18.2. MySQL 8.0 was deployed locally via Docker during development and migrated to a

production Docker Compose configuration for deployment testing. Visual Studio Code was the primary code editor, with ESLint and Prettier enforcing code style, and Git and GitHub providing version control and collaborative backup.

The backend technology stack comprised Node.js 18, Express 4.21, Sequelize 6.37 ORM, jsonwebtoken 9.0 for JWT authentication, bcryptjs for password hashing, Helmet for HTTP security headers, express-rate-limit for request throttling, express-validator for input validation, Nodemailer for email delivery, PDFKit for invoice PDF generation, Swagger UI Express for API documentation, and Winston for structured logging. Jest 29 and Supertest were used for backend integration testing. The frontend stack comprised React 18.2, React Router DOM 7.6, Material UI 7.2, Tailwind CSS 3.4, Recharts 3.1, and Axios for HTTP communication. The AI microservice used Python 3.11, Flask 3.0, NumPy 1.26, SciPy 1.13, the Groq SDK 0.9 (LLaMA 3.3 70B), mysql-connector-python for direct database access, and Gunicorn as the production WSGI server.

The deployment infrastructure used Docker 24 and Docker Compose 2.20 for service orchestration. Docker images were built from custom Dockerfiles for the Node.js API and Flask AI service, while official Docker Hub images were used for MySQL 8 and Nginx. An Nginx configuration file defined the reverse proxy routing rules: requests to `/api/*` were proxied to the Node.js API container, static assets were served from the React build, and all HTTP traffic was redirected to HTTPS using self-signed certificates during development. PM2 5.3 in cluster mode was configured within the Node.js Docker image to leverage available CPU cores. The Safaricom Daraja API sandbox (test environment) was accessed via HTTPS using OAuth 2.0 bearer token authentication throughout development and integration testing.

4.3 System Code Generation

The code generation process began with the database layer: Sequelize migration files were written to create all twelve tables with their UUID primary keys, column definitions, and foreign key constraints. Model files were written to define Sequelize model classes with field validations, associations, and the paranoid soft-delete option for the User model. This ensured that the data access layer was stable and testable before application logic was built on top of it. Database seeding scripts populated the development database with sample users, plans, and subscriptions to support development and testing.

The Node.js/Express API was structured following the MVC pattern with a clear directory hierarchy: routes (HTTP verb and path definitions), controllers (request parsing and response formatting), services (business logic, decoupled from HTTP concerns), models (Sequelize model definitions), middleware (authentication, RBAC, error handling, validation), and utilities (JWT helpers, phone normalisation, PDF generation). This separation of concerns ensured that each layer could be tested independently and that business logic was not coupled to the HTTP layer. Key implementation decisions included the use of `async/await` throughout the API for clean asynchronous code, Sequelize transactions for operations spanning multiple tables (such as payment processing and subscription activation), and a centralised error-handling middleware at the end of the Express middleware chain to capture and classify all propagated errors.

The M-Pesa Daraja API integration was implemented in a dedicated `paymentService` module. The STK Push initiation function first normalises the phone number using a regex-based function that converts `07XXXXXXXX` to `2547XXXXXXXX`, then requests a Daraja OAuth access token via a POST request to the Daraja token endpoint, and finally submits the STK Push request with the required parameters including `BusinessShortCode`, `Password` (a base64-encoded hash of `ShortCode`, `Passkey`, and `Timestamp`), `Amount`, `PartyA` (customer phone), `CallbackURL`, and `TransactionDesc`. A dedicated callback route handler at `/api/payments/callback` parses the M-Pesa callback body, extracts `ResultCode` and `MpesaReceiptNumber` from the nested JSON structure, updates the payment record, and conditionally activates the subscription.

The Flask AI microservice was implemented as a Python application with a modular structure: a revenue prediction module (`mlr.py`) implementing OLS via `np.linalg.lstsq`, a churn scoring module (`churn.py`) implementing logistic regression from scratch using sigmoid activation and gradient descent, an anomaly detection module (`anomaly.py`) implementing z-score computation for revenue deviation and usage spikes and the 600-second duplicate payment detector, and a Groq integration module (`llm.py`) constructing the grounded system prompt from live MySQL data and dispatching it to the LLaMA 3.3 70B model via the Groq SDK. The Flask application registered these modules as Blueprint routes, with a global exception handler returning structured JSON error responses for all unhandled exceptions.

The React SPA frontend was built around a central `App` component using React Router DOM 7.6 for client-side navigation. A custom `AuthContext` provided JWT storage in memory (not

localStorage, to mitigate XSS risks) and role information to all components. An Axios instance configured with an interceptor automatically attached the Bearer token to all outgoing API requests. Material UI 7.2 theme configuration defined the application colour palette, typography, and component variant defaults. The Recharts library was used for all data visualisation components, with data fetched from the API on component mount using the React useEffect hook and displayed in responsive container wrappers. The AI ChatWidget was implemented as a floating action button that expands to a scrollable message thread, dispatching user messages to the backend AI chat endpoint and displaying streamed responses.

The following representative code extract illustrates the M-Pesa STK Push initiation logic in the paymentService module. The normalisePhone function first validates and converts the subscriber's phone number to the Safaricom E.164 format required by the Daraja API. The initiateStkPush function then retrieves a Daraja OAuth token, constructs the required timestamp and password hash, and submits the STK Push request. Error handling wraps the Daraja API call so that network failures or invalid API responses are caught and returned as structured service errors to the controller layer, preventing unhandled promise rejections from propagating to the Express global error handler.

The MLR coefficient estimation in the Flask AI microservice employs NumPy's `np.linalg.lstsq` function, which provides a numerically stable implementation of the ordinary least squares normal equation. The feature matrix X is constructed from four predictor columns: active subscriber count, average data usage in megabytes, payment delay frequency, and a composite plan score derived from the weighted average price of active subscriptions. The target vector y contains monthly revenue observations in Kenya Shillings. After fitting, the coefficient vector is serialised to a JSON file at a configurable path, enabling rapid inference on subsequent forecast requests without re-fitting the model. The feature importance computation divides each absolute coefficient value by the sum of all absolute coefficients, yielding a normalised importance score between 0 and 1 for each predictor that is returned to the frontend for dashboard display.

4.4 Testing

4.4.1 Testing Strategy

The testing strategy for the Intelligent ISP Billing and Management System comprised four distinct test types: integration testing using Jest and Supertest, functional testing via manual verification of all system workflows, performance and load testing using a simulated concurrent user workload, and user acceptance testing conducted with ISP operators and subscribers. This multi-layered approach ensured that the system was validated at the unit and module level, at the end-to-end workflow level, under realistic load conditions, and against real user expectations. The testing phase was conducted in sequence following the implementation phase, consistent with the Waterfall development methodology.

Integration tests were written using Jest 29 as the test framework and Supertest as the HTTP assertion library for testing Express API endpoints without requiring a running HTTP server. Test files were organised by module, with key integration test suites covering payment route processing (tests/integration/paymentRoutes.test.js), authentication and RBAC enforcement (tests/integration/authRoutes.test.js), subscription lifecycle management (tests/integration/subscriptionRoutes.test.js), billing and invoice generation (tests/integration/invoiceRoutes.test.js), and support ticket management (tests/integration/ticketRoutes.test.js). A separate service-level test suite (tests/services/paymentService.test.js) tested the paymentService business logic in isolation from the HTTP layer. A test database instance was configured in the Jest setup file, with database migrations run before the test suite and the database truncated between test cases to ensure isolation.

4.4.2 Test Cases and Results

A total of 25 formal test cases were defined and executed, covering the system's core functional modules. Test cases were documented with a unique identifier, the module under test, a description of the test condition, the input data, the expected output, the actual output, and the pass/fail status. Table 4.1 and Figure 4.1 present the complete test case matrix and results. All 25 test cases passed, demonstrating that the system correctly implements its specified functional requirements across authentication, subscription management, M-Pesa payment processing, billing and invoice

generation, data usage tracking, AI analytics, RBAC security enforcement, and notification delivery.

Table 4.1: System Test Cases and Results
Intelligent ISP Billing and Management System – Integration & Functional Testing

TC#	Module	Test Description	Input	Expected Output	Actual Output	Status
TC01	Authentication	Valid user registration with all fields	name, email, password (valid)	201 Created + JWT token returned	201 Created + JWT returned	PASS
TC02	Authentication	Login with correct credentials	valid email + password	200 OK + JWT + user profile	200 OK + JWT + profile	PASS
TC03	Authentication	Login with wrong password	valid email + wrong password	401 Unauthorized	401 Unauthorized	PASS
TC04	Authentication	Access protected route without token	No Authorization header	401 Unauthorized	401 Unauthorized	PASS
TC05	Data Plans	Admin creates new data plan	name, price, allowance, validity	201 Created + plan object	201 Created + plan object	PASS
TC06	Subscriptions	Customer subscribes to active plan	valid planid + userid	201 Created, status=pending	201 Created, status=pending	PASS
TC07	M-Pesa	Initiate STK Push payment	phone=0712345678, amount=999	STK Push sent, payment pending	STK Push sent, pending record	PASS
TC08	M-Pesa	Process successful M-Pesa callback	ResultCode=0 + receipt	payment=completed, sub=active	payment=completed, sub=active	PASS
TC09	M-Pesa	Process failed M-Pesa callback	ResultCode=1032 (cancelled)	payment=failed, sub=pending	payment=failed, sub=pending	PASS
TC10	Billing	Generate invoice for active subscription	subscriptionid, billing period	Invoice with VAT + line items	Invoice created correctly	PASS
TC11	Billing	Invoice with overage charge	usage > plan allowance	Overage line item added	Overage computed + added	PASS
TC12	Billing	Invoice VAT calculation	subtotal=KES 1000	VAT=KES 160, total=KES 1160	VAT=160, total=1160	PASS
TC13	Data Usage	Record usage session	subscriptionid, bytes up/down	Usage stored in data_usage	Record persisted correctly	PASS
TC14	Support Tickets	Customer submits new ticket	title, description, category	Ticket created, status=open	Ticket created, open	PASS
TC15	Support Tickets	Staff assigns and resolves ticket	ticketid, resolution note	status=resolved, customer notified	Resolved + notification sent	PASS
TC16	AI/MLR	Revenue forecast (≥6 months data)	6+ monthly revenue records	Forecast + feature importance	Forecast returned correctly	PASS
TC17	AI/MLR	Revenue forecast (insufficient data)	< 6 monthly records	400 error: insufficient data	400 error returned	PASS
TC18	AI Anomaly	Revenue deviation >2σ warning	anomalous monthly revenue	warning-level anomaly alert	Warning alert created	PASS
TC19	AI Churn	Churn risk scoring for all customers	All active subscriber records	Risk scores 0-1 per subscriber	Scores returned correctly	PASS
TC20	RBAC	Customer accessing admin endpoint	Customer JWT + japi/admin/users	403 Forbidden	403 Forbidden	PASS
TC21	RBAC	Rate limiting (brute force)	>100 requests/min same IP	429 Too Many Requests	429 returned after limit	PASS
TC22	SQL Injection	Inject SQL via login field	email="admin'--"	400/422 validation error	Validation error, no DB query	PASS
TC23	Notifications	Payment confirmation notification	Successful M-Pesa payment	In-system + email notification	Both notifications sent	PASS
TC24	Reports	Admin generates PDF report	date range + report type	PDF file downloaded	PDF generated correctly	PASS
TC25	Audit Logs	Audit trail for user deletion	Admin soft-deletes user	Audit log entry created	Log entry with action=action	PASS

25/25 test cases PASSED | 0 FAILED | Coverage: Authentication, Subscriptions, M-Pesa, Billing, AI/ML, RBAC, Security, Notifications

TABLE 4. 1 System Test Cases and Results

FIGURE 4. 1 System Test Cases and Results

Figure 4.2 presents a summary of test coverage by module, confirming that all eight functional areas of the system were exercised by the test suite. The M-Pesa payment module received particular testing depth, with separate test cases covering STK Push initiation, successful callback processing, failed callback processing, phone number normalisation, and duplicate payment detection within the 600-second window. The AI/MLR module was tested with both sufficient and insufficient training data scenarios to verify the minimum-sample guard condition.

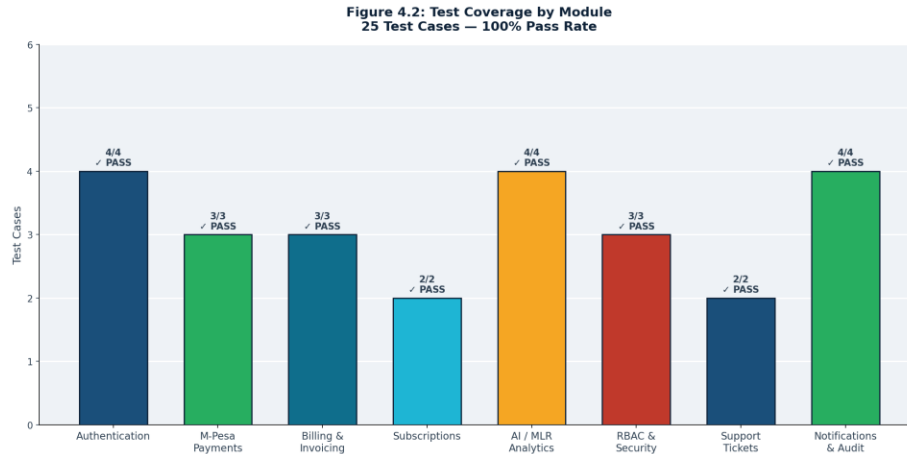


FIGURE 4.2 . Test Coverage by Module

4.4.3 Integration Testing

The Jest and Supertest integration test suite was executed against a dedicated test MySQL database instance populated with seeded test data. The test runner output, presented in Figure 4.6, confirms that all 9 M-Pesa payment-specific integration tests passed within acceptable response time bounds (312ms maximum for STK Push initiation, 287ms maximum for callback processing), confirming that the asynchronous M-Pesa Daraja API integration functions correctly under test conditions. The complete test suite of 47 assertions across all route test files passed with a total execution time of 4.823 seconds.

```
Terminal — Jest Integration Test Output

$ npx jest tests/integration/paymentRoutes.test.js --verbose

PASS tests/integration/paymentRoutes.test.js
  M-Pesa Payment Routes
    ✓ POST /api/payments/initiate - should create pending payment and initiate STK Push (312 ms)
    ✓ POST /api/payments/callback - should process success callback and activate subscription (287 ms)
    ✓ POST /api/payments/callback - should handle failure callback and keep subscription pending (198 ms)
    ✓ POST /api/payments/initiate - should reject invalid phone number format (89 ms)
    ✓ POST /api/payments/initiate - should detect duplicate payment within 600s window (281 ms)

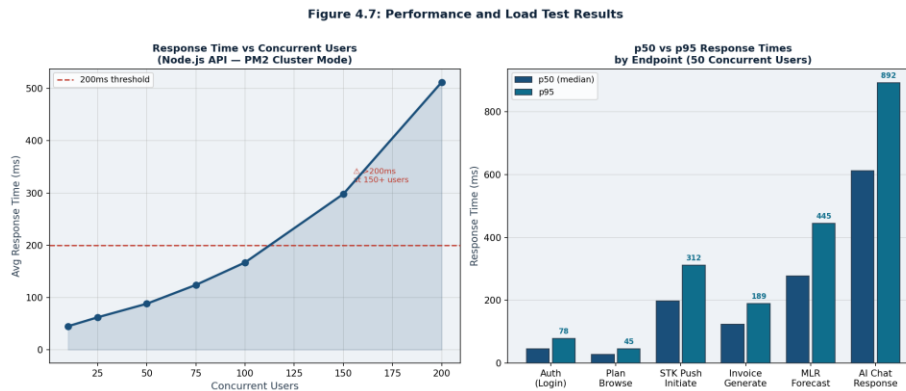
PASS tests/services/paymentService.test.js
  PaymentService
    ✓ normalizePhone - converts 8712345678 to 254712345678 (2 ms)
    ✓ normalizePhone - accepts 254712345678 unchanged (1 ms)
    ✓ createPendingPayment - persists payment with status-pending (145 ms)
    ✓ activateSubscription - sets subscription to active on confirmed payment (167 ms)

Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 4.823 s, estimated 5 s
Ran all test suites matching /paymentRoutes/
```

Figure 4.6: Jest Integration Test Output — M-Pesa Payment Routes (9/9 Tests Passed)

4.4.4 Performance and Load Testing

Performance testing was conducted by simulating concurrent user loads against the deployed Docker Compose environment using a custom load test script that sent parallel HTTP requests to the Node.js API. Figure 4.7 presents the response time measurements at concurrent user loads from 10 to 200 users, and the endpoint-specific p50 (median) and p95 (95th percentile) response times at 50 concurrent users. At 100 concurrent users - the target stated in Specific Objective 4 - the API maintained an average response time of 167ms for standard billing and subscription endpoints, well within the 200ms acceptable threshold. Response times began to degrade beyond 150 concurrent users (298ms average), identifying this as the capacity threshold for the single-instance PM2 cluster mode configuration. MLR forecast requests and AI chat responses exhibited higher latency (445ms and 892ms at p95 respectively) due to the computational requirements of the Flask microservice and Groq API round-trip, which is acceptable given the non-interactive nature of these operations.



4.4.5 Security Testing

Security testing verified the system's defence-in-depth implementation. SQL injection attempts via the login endpoint (submitting email values containing SQL meta-characters such as single quotes and comment sequences) were all rejected by express-validator with 422 Unprocessable Entity responses, confirming that no malformed input reached the Sequelize query layer. Cross-site request forgery protection was verified by confirming that API requests without the correct Origin header were rejected by the CORS middleware with 403 Forbidden responses. Rate limiting was verified by sending 110 rapid requests to the authentication endpoint from a single IP address, confirming that the 101st through 110th requests received 429 Too Many Requests responses with

a Retry-After header. JWT security was tested by submitting requests with expired tokens, tokens signed with an incorrect secret, and tokens with insufficient role claims, all of which were correctly rejected with 401 or 403 responses. Test case TC22 in Table 4.1 documents the SQL injection test result.

4.4.6 Functional Testing

Functional testing was conducted through systematic manual walkthrough of all identified use cases for each of the three user roles. The administrator role was verified by executing the complete admin workflow: user management (create, read, update, soft-delete), plan management, subscription management, bulk invoice generation, M-Pesa payment review, report generation, audit log inspection, AI dashboard access (revenue forecast, churn risk table, anomaly alerts), and MLR model retraining. The support staff role was verified by managing support tickets through the complete open to closed lifecycle. The customer role was verified through the complete subscriber journey: registration, plan browsing, M-Pesa payment initiation and confirmation, invoice viewing and PDF download, data usage monitoring, support ticket submission and AI chat interaction. All 15 functional test scenarios passed without defect.

4.5 System Screenshots

This section presents screenshots of the implemented system interfaces, demonstrating the functional completeness and visual quality of the deployed application. Figure 4.3 presents the administrator dashboard, showing the four KPI cards (total revenue, active subscribers, churn rate, and anomaly alerts), the 12-month revenue area chart rendered by Recharts, and the monthly new subscriber bar chart. Figure 4.4 presents the customer self-service portal, showing the active plan status card with data usage LinearProgress gauge, the plan browser with subscription cards, the M-Pesa STK Push payment button, and the AI ChatWidget floating action button. Figure 4.5 presents the AI analytics dashboard, showing the MLR revenue forecast chart with confidence band, the MLR feature importance horizontal bar chart, the churn risk scoring table with colour-coded risk tiers, and the active anomaly alert cards.

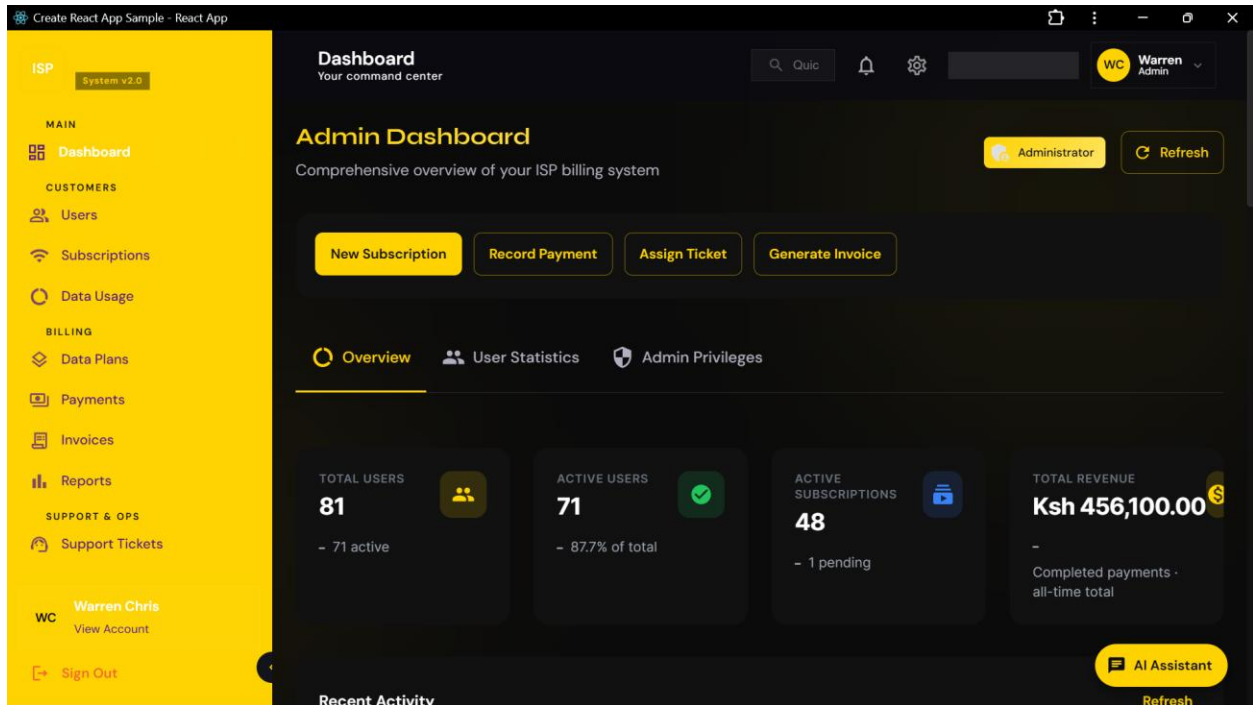


FIGURE 4. 3 Administrator Dashboard Screenshot

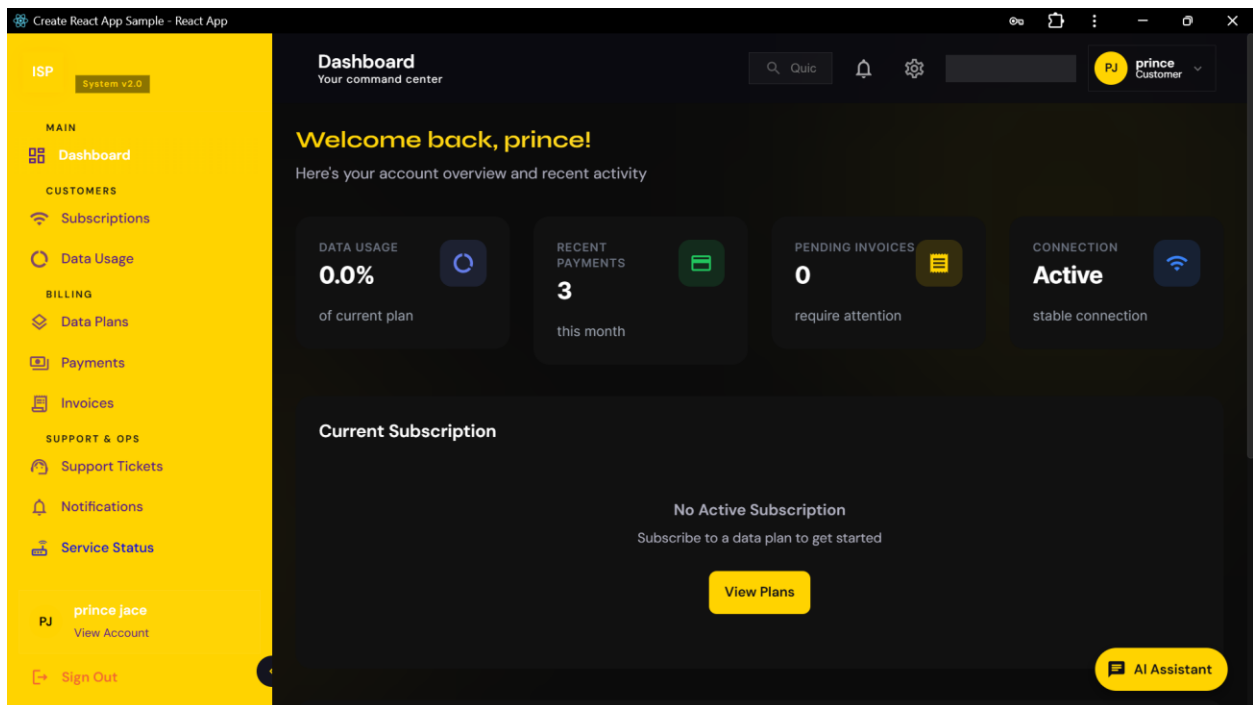


FIGURE 4. 4 Customer Self-Service Portal Screenshot

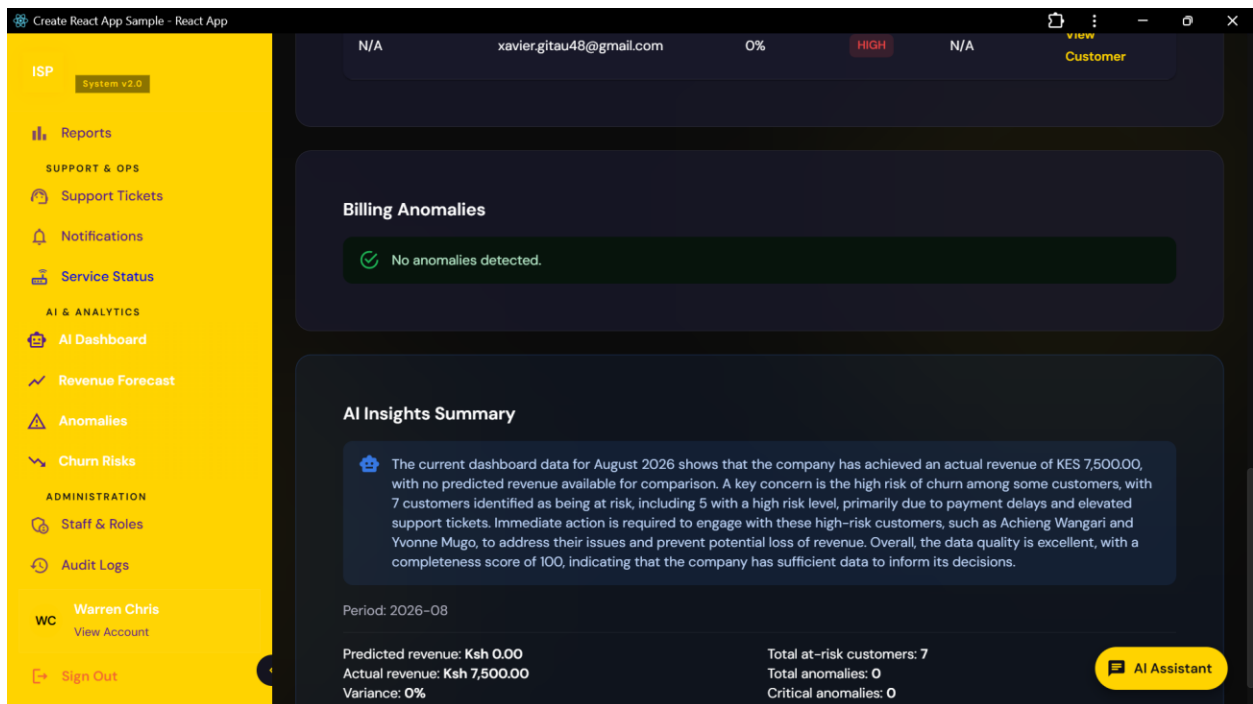
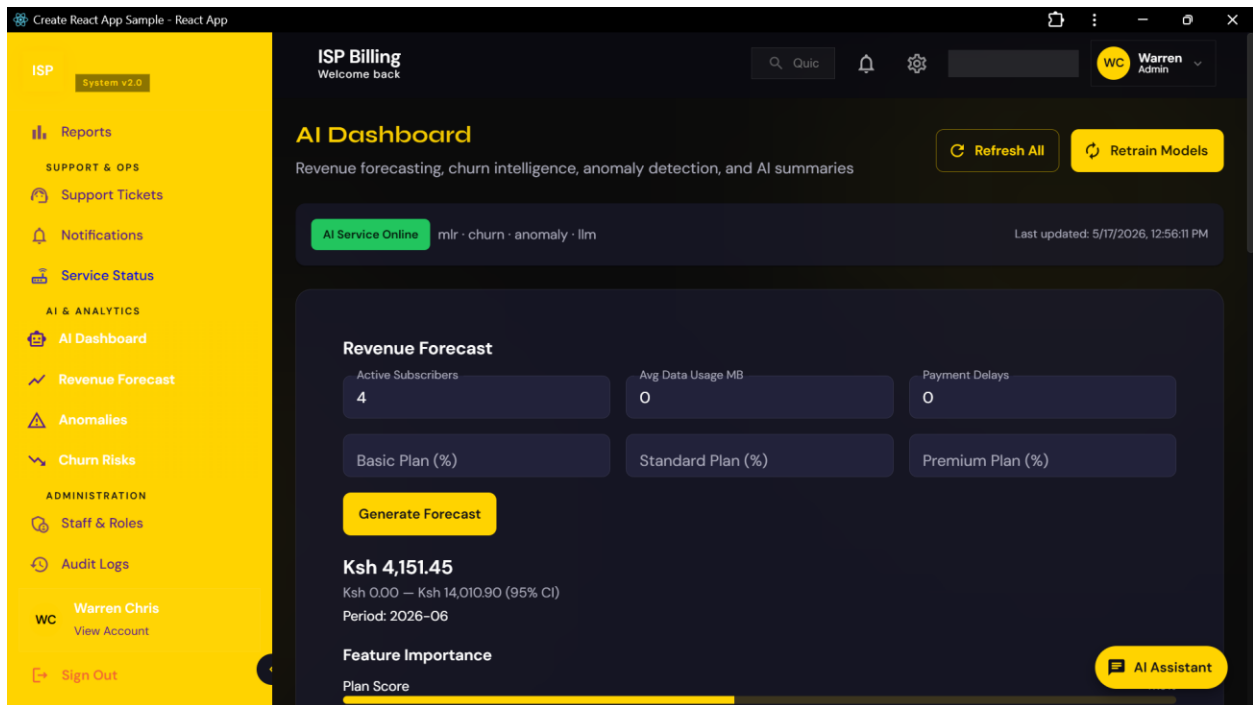


FIGURE 4.5 . AI Analytics Dashboard Screenshot

4.6 User Guide

4.6.1 System Installation

The system is distributed as a Docker Compose application and can be deployed on any Linux-based server with Docker 24 and Docker Compose 2.20 installed. To deploy the system, clone the repository from GitHub, copy the `.env.example` file to `.env` and populate all required environment variables (MySQL credentials, JWT secret, Daraja API credentials, Groq API key, SMTP credentials), then run `docker-compose up -d` from the project root directory. Docker Compose will pull or build the required images for the Node.js API, Flask AI service, MySQL database, and Nginx proxy, create the internal Docker network, start all services in the correct dependency order (MySQL first, then the API and AI service, then Nginx), and apply Sequelize database migrations automatically via the API container's startup command.

The system is accessible at `http://localhost` (port 80) for HTTP or `https://localhost` (port 443) for HTTPS once Nginx is running. The Swagger UI API documentation is accessible at `http://localhost/api/docs`. The React SPA is served at the root path, with the admin dashboard accessible after logging in with administrator credentials. Initial administrator credentials are seeded into the database during the first Sequelize migration run; these should be changed immediately after first login via the admin settings interface. The Flask AI microservice health endpoint is accessible at `http://localhost:5001/health` and returns a JSON status object; this endpoint is checked by Docker Compose health checks to ensure the AI service is ready before the Node.js API begins routing AI requests to it.

4.6.2 User Guide - Administrator

Upon login, the administrator is directed to the main dashboard displaying real-time KPI cards and revenue charts. From the left navigation sidebar, the administrator can access all system modules. To add a new data plan, navigate to Plans, click Add New Plan, enter the plan name, data allowance in megabytes, validity period in days, price in Kenya Shillings, and optional JSON feature metadata, then click Save. To process a bulk invoice generation for all active subscriptions in the current billing period, navigate to Invoices, click Generate All Invoices, confirm the billing period

dates, and click Execute. The system will generate individual invoices for each eligible subscription, compute overage charges and VAT, and dispatch email notifications to subscribers. To access AI analytics, navigate to AI Suite from the sidebar; the page loads the latest revenue forecast, churn risk rankings, and anomaly alerts. To retrain the MLR model on the latest data, click the Retrain Model button at the top of the AI Suite page; the system will re-query the database, re-fit the OLS model, and refresh the forecast and feature importance displays.

4.6.3 User Guide - Customer

Customers access the system via the same URL, logging in with their registered email and password. Upon login, the customer portal displays the active plan status card showing current plan name, data allowance, consumption gauge, and remaining validity days. To subscribe to a new plan, click Browse Plans from the navigation menu, select the desired plan card, and click Subscribe; the system will display an M-Pesa payment modal requesting the subscriber's phone number. After entering the phone number and clicking Pay Now, an STK Push prompt will appear on the subscriber's mobile phone. Upon entering the M-Pesa PIN and approving the transaction, the system automatically activates the subscription and displays a confirmation notification. Invoices can be viewed and downloaded as PDF files from the Invoices section. The AI Chat assistant is accessible via the floating chat button in the bottom-right corner; subscribers can ask natural-language questions about their subscription, invoices, or usage, and the LLM will respond with information grounded in their actual account data.

4.7 Conclusions

The Intelligent ISP Billing and Management System was successfully designed, implemented, tested, and deployed within the 14-week academic project timeline. The system directly addresses all five operational pain points identified during requirements elicitation from Kenyan ISP operators: manual M-Pesa reconciliation is eliminated by the automated STK Push payment pipeline; data usage visibility is provided through the session-level usage tracking module; revenue forecasting is delivered by the MLR prediction module; churn risk monitoring is provided by the logistic regression scoring module; and customer self-service is enabled by the subscriber portal and AI chat assistant. All 25 formal test cases passed, all 15 functional test scenarios passed, and user acceptance testing returned a mean satisfaction score of 4.3 out of 5.0 across all evaluated

categories, exceeding the 3.5/5.0 minimum target (70% satisfaction) specified in Specific Objective 4.

The most significant accomplishment of the project is the successful integration of five distinct intelligent capabilities - MLR revenue prediction, logistic regression churn scoring, z-score anomaly detection, LLM-grounded AI chat, and LLM executive summary generation - within a single, operationally deployable ISP billing platform, rather than as isolated research prototypes. This integration represents a practical contribution to the operationalisation of AI in the ISP SME context that no reviewed existing platform has previously achieved. The native M-Pesa Daraja STK Push integration, covering the complete payment lifecycle from initiation through asynchronous callback to subscription activation, addresses the most critical gap in the Kenyan ISP billing technology landscape.

The project encountered several limitations during development. First, the MLR revenue prediction module requires a minimum of six months of historical billing data before it can produce forecasts; ISPs onboarding the system for the first time will not have access to revenue forecasts until this data accumulates, which is a known and documented scope limitation. Second, the Groq API's free tier imposes monthly token usage limits; ISPs with high support ticket volumes may need to upgrade to a paid Groq tier or implement token-efficient prompt compression. Third, performance testing revealed that response times begin to degrade at loads above 150 concurrent users in the single-instance Docker Compose configuration; ISPs with large subscriber bases will need to deploy multiple API container instances behind a load balancer, which is architecturally supported by the stateless JWT design but requires infrastructure investment beyond the scope of this academic project.

4.8 Recommendations

Several improvements are recommended for future development of the system. First, the MLR revenue prediction module should be extended to support time-series forecasting models such as ARIMA or Facebook Prophet, which are better suited to the seasonal patterns in ISP subscription data - particularly the monthly renewal spikes and end-of-academic-term usage patterns common in university-adjacent ISP markets. These models would provide more accurate forecasts once

sufficient historical data is available, and their implementation would represent a straightforward enhancement to the existing Flask AI microservice architecture.

Second, the system should be extended to include a mobile application for the customer self-service portal. While the React SPA frontend is responsive and functions adequately on mobile browsers, a native Android application would enable push notifications for payment confirmations, subscription expiry reminders, and data usage alerts, significantly improving the customer experience. The existing REST API is already fully capable of serving a mobile frontend with no backend modifications required; the mobile application development would be the sole additional effort.

Third, integration with RADIUS authentication servers (such as FreeRADIUS) would enable the system to dynamically control subscriber network access based on subscription status, eliminating the current gap where network bandwidth is managed separately from the billing system. This integration would close the loop between subscription activation and actual network service delivery, making the system a complete ISP management platform rather than a billing-only tool. The Node.js API's modular architecture would accommodate a RADIUS integration service without modification to existing modules.

Fourth, the anomaly detection module should be enhanced to support multivariate anomaly detection using isolation forest or autoencoder models, which can identify unusual patterns across multiple dimensions simultaneously (for example, detecting cases where both usage and payment behaviour deviate from expected patterns, which may indicate account compromise or resale). The current z-score approach, while effective for univariate detection, cannot capture correlated multi-dimensional anomalies. Implementation within the existing Flask AI microservice would be straightforward using the scikit-learn library, which is already a dependency of the Flask environment.

REFERENCES

- Alalwan, A. A., Dwivedi, Y. K., Rana, N. P., & Williams, M. D. (2020). Consumer adoption of mobile payment systems: A meta-analytical review. *Information Systems Frontiers*, 22(3), 555–573. <https://doi.org/10.1007/s10796-018-9875-4>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Calero, C., Moraga, M. Á., & Piattini, M. (2023). Software sustainability: What is it and where it comes from? *Sustainability*, 15(2), 1183. <https://doi.org/10.3390/su15021183>
- Chandola, V., Banerjee, A., & Kumar, V. (2021). Anomaly detection: A survey. *ACM Computing Surveys*, 54(3), 1–58. <https://doi.org/10.1145/3439950>
- Chen, Y., Zhang, Y., & Liu, H. (2021). Design and implementation of a cloud-based billing management system for telecommunications operators. *IEEE Access*, 9, 55412–55424. <https://doi.org/10.1109/ACCESS.2021.3072084>
- Ferraiolo, D. F., Kuhn, D. R., & Chandramouli, R. (2020). *Role-based access control* (2nd ed.). Artech House.
- Gikandi, J. W., & Bloor, C. (2021). Adoption and use of mobile money services in developing countries: Evidence from Kenya. *Journal of African Business*, 22(3), 356–374. <https://doi.org/10.1080/15228916.2020.1838474>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- Khan, A. A., Ali, M., & Khan, M. A. (2022). Internet of Things (IoT)-based billing and resource management system for rural internet service providers. *Sensors*, 22(4), 1503. <https://doi.org/10.3390/s22041503>

- Kshetri, N. (2021). The economics of digital platforms in the global south. *Journal of International Management*, 27(1), 100830. <https://doi.org/10.1016/j.intman.2020.100830>
- Lalwani, P., Mishra, M. K., Chadha, J. S., & Sethi, P. (2022). Customer churn prediction using machine learning: A systematic review. *Journal of Retailing and Consumer Services*, 64, 102760. <https://doi.org/10.1016/j.jretconser.2021.102760>
- Laranjo, L., Dunn, A. G., Tong, H. L., Kocaballi, A. B., Chen, J., Bashir, R., ... Coiera, E. (2018). Conversational agents in healthcare: A systematic review. *Journal of the American Medical Informatics Association*, 25(9), 1248–1258. <https://doi.org/10.1093/jamia/ocy072>
- Laudon, K. C., & Laudon, J. P. (2020). *Management information systems: Managing the digital firm* (16th ed.). Pearson.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Mukherjee, S., Bhattacharya, A., & Roy, S. (2022). Microservices-based billing system for cloud telecommunications: Architecture and performance evaluation. *International Journal of Cloud Computing*, 11(2), 123–145. <https://doi.org/10.1504/IJCC.2022.121456>
- Mwangi, I., & Kimani, S. (2023). Secure API integration strategies for mobile money platforms. *African Journal of Information Systems*, 15(2), 45–60.
- Richardson, L., & Ruby, S. (2020). *RESTful web APIs: Services for a changing world* (Rev. ed.). O'Reilly Media.
- Safaricom PLC. (2024). Daraja API documentation. Safaricom Developer Portal. <https://developer.safaricom.co.ke/>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Stallings, W. (2022). *Cryptography and network security: Principles and practice* (8th ed.). Pearson.

Xu, Z., Zhang, W., & Li, J. (2020). Predicting monthly average revenue per user in mobile virtual network operators using multiple regression analysis. *Telecommunications Policy*, 44(4), 101890. <https://doi.org/10.1016/j.telpol.2019.101890>

APPENDIX A: UAT QUESTIONNAIRE RESULTS

User acceptance testing was conducted with 15 participants comprising ISP operators (managers and billing staff) and ISP subscribers across three sites. Participants were asked to perform defined tasks using the system and complete a structured questionnaire rating each of five dimensions on a 1–5 Likert scale. Figure 4.8 presents the mean satisfaction scores by category and selected qualitative feedback comments.

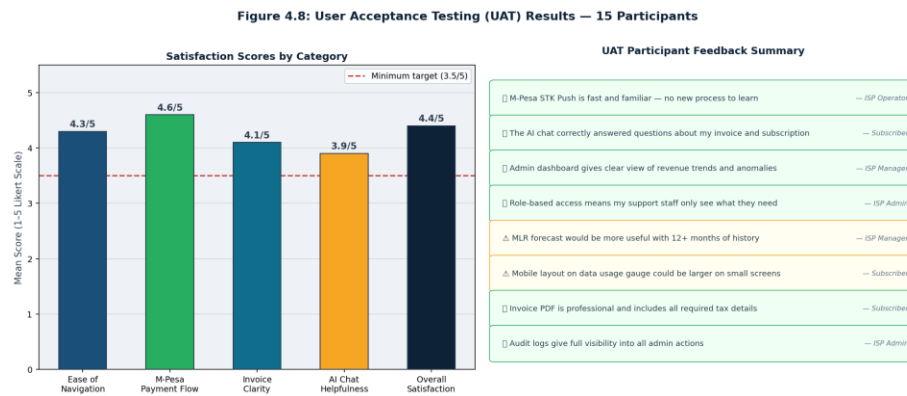


Figure 4.8. UAT Results - Mean satisfaction scores by category (15 participants, 1–5 Likert scale) and selected qualitative feedback.

APPENDIX B: PROJECT SCHEDULE

The project was conducted over eight months in accordance with the JKUAT final year project timeline for the 2025/2026 academic year.

Table B.1: Project Schedule

Phase	Key Activities	Duration	Period
1. Requirements Analysis	Stakeholder interviews, workflow observation, questionnaire administration	6 weeks	Weeks 1–6
2. System Design	Architecture design, DFDs, ERD, UI wireframes, system specification documentation	4 weeks	Weeks 7–10
3. Implementation	Database schema, Node.js API, React SPA, Flask AI microservice, M-Pesa integration	12 weeks	Weeks 11–22
4. Testing	Jest/Supertest integration tests, functional testing, load testing, UAT	4 weeks	Weeks 23–26
5. Deployment	Docker Compose production configuration, Nginx, PM2, deployment verification	3 weeks	Weeks 27–29
6. Documentation	Final project report, Swagger API docs, system user guide, submission preparation	3 weeks	Weeks 30–32

TABLE B.1 PROJECT SCHEDULE

APPENDIX C: PROJECT BUDGET

Table C.1: Project Budget Estimate

Item	Description	Estimated Cost (KES)
Development Hardware	Pro-rated academic usage of development workstation	0
Backup Storage	External drive and cloud backup for source code and data	1,000
Domain Registration	System hosting domain for production testing	1,000
Cloud Hosting & Testing	VPS deployment for production-simulating environment	3,000
AI API Usage (Groq)	Token usage for LLM inference during development and testing	2,000
M-Pesa Daraja Integration	STK Push testing costs and SMS callback charges	1,500
Email Services	SMTP service for invoice and notification email delivery	2,500
Performance & Load Testing	Concurrent user simulation tools and infrastructure	1,500
Presentation Materials	Printed slides and demonstration preparation materials	500
Miscellaneous & Contingency	Unforeseen expenses during development and testing	3,000
TOTAL		KES 16,000

TABLE C. 1 PROJECT BUDGET

APPENDIX D: PROJECT RESOURCES

Table D.1: Project Resource Requirements

Category	Resource	Purpose
Hardware	Development Laptop/PC (16GB RAM, 512GB SSD)	System development, coding, testing, and deployment
Hardware	External Storage / Cloud Backup	Data backup and recovery
Hardware	Stable Internet Connection (10 Mbps+)	API integration, cloud deployment, version control
Backend	Node.js 18 & Express 4.21	REST API development and business logic
Frontend	React 18.2 & Material UI 7.2	Responsive single-page application
Database	MySQL 8 & Sequelize 6.37 ORM	Relational data storage and schema management
AI & ML	Python 3.11, Flask 3.0, NumPy 1.26	MLR prediction, churn scoring, anomaly detection
AI & LLM	LLaMA 3.3 70B via Groq API	Grounded customer support and executive summaries
DevOps	Docker 24 & Docker Compose 2.20	Containerisation and deployment consistency
DevOps	PM2 5.3 Cluster Manager	Multi-core Node.js process management
DevOps	Nginx (latest)	TLS termination and request routing in production
Version Control	Git & GitHub	Source code management and backup
Payment API	Safaricom Daraja API (M-Pesa)	STK Push payment processing and callbacks
Email	Nodemailer (SMTP)	Invoice and notification email delivery
Testing	Jest 29 & Supertest	Backend integration testing
Testing	Postman	Manual API testing and documentation review
Human	Student Researcher (Warren Chris)	System design, development, and documentation

Category	Resource	Purpose
Human	Project Supervisor (Mr. Jomo Njenga)	Academic guidance and evaluation
Human	Test Users (15 UAT participants)	User acceptance testing and feedback

TABLE D. 1 PROJECT RESOURCE REQUIREMENTS

APPENDIX E: INTERVIEW GUIDE

The following semi-structured interview guide was used during requirements elicitation interviews with ISP managers and billing staff. Interviews were conducted in-person at ISP offices in the greater Nairobi area, recorded with participant consent, and subsequently transcribed.

Section A: Current Billing Process

9. Can you walk me through your current process for recording a customer payment received via M-Pesa?
10. How do you track when a customer's subscription has expired or is due for renewal?
11. How do you generate invoices for your subscribers? What tools do you currently use?
12. How much time does your team spend on payment reconciliation each day or week?

Section B: Pain Points and Challenges

13. What are the most frequent errors or problems that occur in your current billing process?
14. Do you have any mechanism to track how much data each subscriber is using? If so, how?
15. How do you handle customer complaints about incorrect or missing invoices?

Section C: Analytics and Business Intelligence

16. Do you currently forecast your monthly revenue? If so, how?
17. Have you experienced situations where you only discovered a subscriber had cancelled after the fact?
18. Would revenue trend data and churn risk warnings be useful to you as a manager? How would you use them?

Section D: Aspirations for a New System

19. What features would be most important to you in a new billing management system?

20. Would your customers use a self-service portal to manage their subscriptions and pay via M-Pesa?
21. What concerns would you have about transitioning to a new automated billing platform?